Szakdolgozat

Miskolci Egyetem

Interaktív e-learning webalkalmazás fejlesztése

Készítette:

Kiss Tamás László

Programtervező informatikus

Témavezető:

Dr. Tompa Tamás, adjunktus

Miskolc, 2026

Miskolci Egyetem
Gépészmérnöki és Informatikai Kar
Alkalmazott Matematikai Intézeti Tanszék **Szám:**

Szakdolgozat Feladat

Kiss Tamás László (WN6YVX) programtervező informatikus jelölt részére.

A szakdolgozat tárgyköre: Webes alkalmazás fejlesztés

A szakdolgozat címe: Interaktív e-learning webalkalmazás fejlesztése

A feladat részletezése:

Ide kell a feladatkiírásban szereplő szöveget betenni.

(Kisebb tagolás lehet benne, hogy jól nézzen ki.)

Témavezető: Dr. Tompa Tamás, (adjunktus)

A feladat kiadásának ideje:

. . . . . .
szakfelelős

Eredetiségi Nyilatkozat

Alulírott **Kiss Tamás László**; Neptun-kód: `WN6YVX` a Miskolci Egyetem Gépészmérnöki és Informatikai Karának végzős Programtervező informatikus szakos hallgatója ezennel büntetőjogi és fegyelmi felelősségem tudatában nyilatkozom és aláírásommal igazolom, hogy *Interaktív e-learning webalkalmazás fejlesztése* című szakdolgozatom saját, önálló munkám; az abban hivatkozott szakirodalom felhasználása a forráskezelés szabályai szerint történt.

Tudomásul veszem, hogy szakdolgozat esetén plágiumnak számít:

- szószerinti idézet közlése idézőjel és hivatkozás megjelölése nélkül;
- tartalmi idézet hivatkozás megjelölése nélkül;
- más publikált gondolatainak saját gondolatként való feltüntetése.

Alulírott kijelentem, hogy a plágium fogalmát megismertem, és tudomásul veszem, hogy plágium esetén szakdolgozatom visszautasításra kerül.

Miskolc, ... év ..... hó ..... nap

.....

Hallgató

1. A szakdolgozat feladat módosítása

szükséges (módosítás külön lapon)
nem szükséges

....
dátum témavezető(k)

2. A feladat kidolgozását ellenőriztem:

témavezető (dátum, aláírás):
.....
.....
.....

konzulens (dátum, aláírás):
.....
.....
.....

3. A szakdolgozat beadható:

....
dátum témavezető(k)

4. A szakdolgozat ..... szövegoldalt
..... program protokollt (listát, felhasználói leírást)
..... elektronikus adathordozót (részletezve)
.....
..... egyéb mellékletet (részletezve)
.....
tartalmaz.

....
dátum témavezető(k)

5. A szakdolgozat bírálatra

bocsátható
nem bocsátható

A bíráló neve: .....

....
dátum szakfelelős

6. A szakdolgozat osztályzata

a témavezető javaslata: .....
a bíráló javaslata: .....
a szakdolgozat végleges eredménye: .....

Miskolc, ...
a Záróvizsga Bizottság Elnöke

Tartalomjegyzék

1. Bevezetés **1**

2. Koncepció **2**
2.1. Problémakör és motiváció . . . 2
2.2. Hasonló rendszerek áttekintése . . . 3
2.2.1. Moodle . . . 3
2.2.2. Google Classroom . . . 3
2.2.3. Canvas LMS . . . 3
2.2.4. Open edX . . . 4
2.2.5. Összehasonlítás . . . 4
2.3. Felhasznált technológiák . . . 4
2.3.1. Java és a Spring-ökoszisztéma általában . . . 4
2.3.2. Spring Boot . . . 4
2.3.3. Spring Security . . . 5
2.3.4. Spring Data JPA és Hibernate . . . 5
2.3.5. Thymeleaf . . . 6
2.3.6. Apache POI . . . 6
2.3.7. Apache PDFBox . . . 6
2.4. Követelmény-specifikáció . . . 6
2.4.1. Funkcionális követelmények . . . 7
2.4.2. Nemfunkcionális követelmények . . . 7

3. Tervezés **9**
3.1. Rendszerarchitektúra . . . 9
3.1.1. Háromrétegű architektúra . . . 9
3.1.2. Kérés-feldolgozás folyamata . . . 9
3.1.3. Telepítési modell . . . 10
3.2. Adatmodell . . . 10
3.2.1. User . . . 11
3.2.2. Course . . . 11
3.2.3. Lesson . . . 12
3.2.4. Presentation . . . 12
3.2.5. Quiz és Question . . . 12
3.2.6. QuizResult . . . 12
3.2.7. Role . . . 13
3.2.8. Entitáskapcsolatok összefoglalása . . . 13
3.3. Biztonsági tervezés . . . 14
3.3.1. Szerepkör-alapú hozzáférés-vezérlés . . . 14

3.3.2. URL-szintű hozzáférés-vezérlés . . . 14
3.3.3. Jelszókezelés . . . 14
3.3.4. Munkamenet-kezelés . . . 15
3.4. Felhasználói felület tervezése . . . 15
3.4.1. Általános UX-elvek . . . 15
3.4.2. Oldalak és navigációs folyamatok . . . 15
3.4.3. Sablon-hierarchia és töredékek . . . 16
3.5. Kvízrendszer tervezése . . . 16
3.5.1. Többszöri próbálkozás kezelése . . . 16
3.5.2. Ponthatárok és osztályzatok . . . 16
3.5.3. Eredménystatisztikák . . . 16
3.6. Prezentáció-feldolgozás tervezése . . . 16
3.6.1. Támogatott formátumok és konverziós stratégia . . . 16
3.6.2. Megjelenítési megközelítés . . . 18
3.6.3. Tárolási megfontolások . . . 18
3.7. Réteg-struktúra a forráskódban . . . 18
3.7.1. Függőségek iránya . . . 18
3.8. Nézetek (Thymeleaf sablonok) . . . 19

4. Megvalósítás **20**
4.1. Az alkalmazás indítása és konfiguráció . . . 20
4.1.1. A belépési pont . . . 20
4.1.2. Maven függőségek . . . 20
4.1.3. Az `application.properties` konfiguráció . . . 21
4.2. A forráskód szerkezete . . . 21
4.3. Adatmodell megvalósítása . . . 22
4.3.1. A User entitás . . . 22
4.3.2. A Course entitás . . . 23
4.3.3. A Lesson entitás . . . 23
4.3.4. A Presentation entitás . . . 24
4.3.5. A Quiz és Question entitás . . . 25
4.3.6. A QuizResult entitás . . . 25
4.4. Adatinicializálás . . . 26
4.5. Biztonsági konfiguráció . . . 27
4.5.1. A SecurityConfig osztály . . . 27
4.5.2. A CustomUserDetailsService . . . 28
4.6. Adatelérési réteg . . . 30
4.7. Szolgáltatási réteg . . . 31
4.7.1. UserService . . . 31
4.7.2. CourseService . . . 31
4.7.3. LessonService . . . 32
4.7.4. QuizService . . . 32
4.8. Felhasználókezelés megvalósítása . . . 33
4.8.1. Regisztráció és bejelentkezés . . . 33
4.8.2. Irányítópultok . . . 34
4.9. Kurzus- és leckekezelés megvalósítása . . . 35
4.9.1. Kurzuslista keresővel . . . 35
4.9.2. Kurzus létrehozása . . . 36

4.9.3. Lecke létrehozása . . . 37
4.9.4. Kurzusnézet leckékkel . . . 38
4.9.5. Prezentáció-feltöltés leckéhez rendeléssel és duplikáció-szűréssel . 39
4.10. Prezentáció-feldolgozás megvalósítása . . . 40
4.10.1. A PresentationService . . . 40
4.10.2. PowerPoint–PDF konverzió . . . 41
4.10.3. Prezentáció streamelése . . . 43
4.11. Kvízrendszer megvalósítása . . . 44
4.11.1. Kvíz kitöltési és kiértékelési logika . . . 44
4.11.2. Ponthatár-módosítás visszamenőleges hatása . . . 44
4.12. E-mail értesítési rendszer . . . 45
4.13. Nézetek és sablon-rendszer . . . 45
4.13.1. Sablon-struktúra . . . 45
4.13.2. Mobiloptimalizálás és reszponzív nézetek . . . 45
4.13.3. Thymeleaf–Spring Security integráció . . . 45
4.13.4. Statikus erőforrások . . . 46

5. Tesztelés **47**
5.1. Tesztelési stratégia . . . 47
5.2. Automatizált tesztek . . . 47
5.2.1. Alkalmazáskontextus-teszt . . . 47
5.2.2. Regisztrációs és bejelentkezési integrációs teszt . . . 48
5.2.3. Jogosultságvezérlési integrációs teszt . . . 49
5.2.4. Kvíz üzleti logika egységteszt . . . 50
5.3. Manuális funkcionális tesztelés . . . 51
5.3.1. Hitelesítési tesztek . . . 52
5.3.2. Kurzus- és leckekezelési tesztek . . . 53
5.3.3. Prezentáció-feltöltési tesztek . . . 54
5.3.4. Kvíztesztelés . . . 55
5.3.5. Irányítópult tesztek . . . 56
5.3.6. Felhasználókezelési tesztek . . . 56
5.3.7. E-mail értesítési teszt . . . 57
5.4. Biztonsági tesztelés . . . 57
5.4.1. URL-manipulációs kísérletek . . . 57
5.4.2. CSRF-védelem ellenőrzése . . . 57
5.5. Reszponzivitás és böngészőkompatibilitás tesztelése . . . 57
5.6. Ismert korlátok és fejlesztési lehetőségek . . . 58
5.6.1. Ismert korlátok . . . 58
5.6.2. Fejlesztési lehetőségek . . . 59

6. Összefoglalás **60**

Források **61**

1. fejezet

Bevezetés

A digitális oktatás napjainkban egyre nagyobb szerepet kap az egyetemi és vállalati képzésekben egyaránt. Az online tanulási platformok lehetővé teszik, hogy a tanulni vágyók időben és térben rugalmasan sajátítsák el az ismereteket, miközben az oktatók hatékonyan kezelhessék a kurzusanyagokat és nyomon követhessék a résztvevők előrehaladását.

Jelen dolgozat egy Java alapú e-learning webalkalmazás tervezését és megvalósítását mutatja be, amelyet az oktatási és képzési folyamatok modernizálása, valamint széleskörű digitális támogatása céljából fejlesztettem ki. Az alkalmazás neve ME Learning, és célja, hogy egyetlen, könnyen kezelhető felületen keresztül tegye lehetővé a kurzusok kezelését, a prezentációs anyagok megtekintését, valamint az interaktív kvízek kitöltését.

A fejlesztés során különös figyelmet fordítottam a szerepkör-alapú hozzáférésvezérlésre: az adminisztrátorok, oktatók és hallgatók eltérő jogosultságokkal rendelkeznek a rendszerben. Az oktatók kurzusokat hozhatnak létre, PDF és PowerPoint prezentációkat tölthetnek fel, kvízeket szerkeszthetnek, és kezelhetik a beiratkozott felhasználók listáját. A hallgatók böngészhetik a nyilvános kurzusokat, beiratkozhatnak, megtekinthetik az oktatási anyagokat és kitölthetik a kvízeket.

A dolgozat szerkezete a klasszikus szoftverfejlesztési folyamatot követi: a koncepcióalkotást a tervezési döntések bemutatása követi, majd az implementáció részleteit és a tesztelés eredményeit tárgyalom. A fejlesztés Spring Boot keretrendszer segítségével, Thymeleaf sablon-motorral és H2 beágyazott adatbázissal készült.

2. fejezet

Koncepció

2.1. Problémakör és motiváció

Az e-learning, azaz elektronikus tanulás fogalma az 1990-es évek végén jelent meg, és azóta a digitális oktatás egyik meghatározó területévé vált. Az e-learning tágabb értelemben minden olyan oktatási folyamatot magában foglal, amelyet digitális eszközök és hálózati technológiák támogatnak [17]. A számítógépes hálózatok, különösen az internet elterjedésével az e-learning nemcsak az oktatás elérhetőségét növelte, hanem alapvetően megváltoztatta a tudástranszfer módját és sebességét is.

Az utóbbi évtizedben a felsőoktatásban és a vállalati képzések területén egyaránt ugrásszerűen nőtt a webalapú tanulási platformok iránti igény. Allen és Seaman vizsgálatai szerint az online oktatásban részt vevő hallgatók száma évről évre dinamikusan emelkedik az Egyesült Államokban [1]. Ez a tendencia globálisan is megfigyelhető: a COVID-19 pandémia tovább erősítette ezt a folyamatot, mivel a felsőoktatási intézmények kénytelenek voltak hirtelen és teljes körűen digitalizálni oktatási folyamataikat [12]. A járvány rávilágított arra, hogy a rugalmas, helyhez és időhöz nem kötött oktatási platformok nem csupán kiegészítői, hanem esetenként egyenrangú alternatívái lehetnek a hagyományos kontaktoktatásnak.

A Learning Management System (LMS), azaz tanulásirányítási rendszer egy olyan szoftveres infrastruktúra, amely lehetővé teszi oktatási tartalmak létrehozását, kezelését, terjesztését és a tanulói előrehaladás nyomon követését [21]. A piacon elérhető LMS-megoldások jellemzően két nagy csoportba sorolhatók: a kereskedelmi, licenszköteles rendszerek (pl. Blackboard, Canvas) és a nyílt forráskódú platformok (pl. Moodle, Open edX). Mindkét kategória képviselői általában komplex, sok esetben nehezen testreszabható architektúrával rendelkeznek, amelyek kis intézmények vagy egyedi igények esetén aránytalanul nagy erőforrásigényt támasztanak az üzemeltetőkkel szemben.

A ME Learning alkalmazás fejlesztésének kiindulópontját éppen ez a hiányosság adta. A cél egy könnyűsúlyú, könnyen telepíthető és karbantartható alternatíva biztosítása, amely a következő alapvető funkciókat nyújtja: kurzuskezelés, prezentációs anyagok tárolása és böngészőalapú megjelenítése, kvíz-alapú tudásellenőrzés, valamint szerepkör-alapú hozzáférés-vezérlés. Az alkalmazás tervezésekor kiemelt szempont volt, hogy az ne igényeljen dedikált alkalmazásszerver-infrastruktúrát, és a telepítése minimális rendszergazdai előismerettel elvégezhető legyen.

2.2. Hasonló rendszerek áttekintése

A piacon jelenleg számos LMS-megoldás érhető el, amelyek különböző felhasználói igényeket és intézményi méreteket céloznak meg. Az alábbiakban a legelterjedtebb rendszereket mutatom be, kiemelve azokat a tulajdonságokat, amelyek a ME Learning fejlesztési irányvonalát meghatározták.

2.2.1. Moodle

A Moodle (Modular Object-Oriented Dynamic Learning Environment) a világ legelterjedtebb nyílt forráskódú LMS-platformja, amelyet Martin Dougiamas fejlesztett ki 2002-ben, és amelyet mára több mint 300 millió felhasználó vesz igénybe világszerte [15]. A rendszer PHP-ban íródott, és az objektumorientált tervezési elveket követi. Moduláris architektúrájának köszönhetően szinte bármilyen oktatási igényre kiterjeszthető: fórumok, feladatkezelés, tesztelés, haladáskövetés, videókonferencia-integráció mind elérhetők bővítményként.

Az MVC (Model-View-Controller) tervezési mintát alkalmazó rendszer képes SCORM- és IMS-szabványú tartalmak kezelésére is, ami széleskörű interoperabilitást biztosít [7]. Ugyanakkor a Moodle telepítése, konfigurálása és karbantartása adminisztrátori szaktudást igényel: szükséges hozzá webszerver (pl. Apache vagy Nginx), PHP-értelmező, valamint relációs adatbázis-kezelő rendszer (MySQL, PostgreSQL vagy MSSQL). Mindezek miatt a ME Learning célközönsége számára a Moodle túlméretezett és túl erőforrásigényes megoldás lenne.

2.2.2. Google Classroom

A Google Classroom 2014-ben jelent meg a Google Workspace for Education csomag részeként, és mára az egyik legszélesebb körben használt oktatási platform lett, különösen az alap- és középfokú oktatásban [10]. Legfőbb előnye az egyszerűsége és a Google-ökoszisztémával való szoros integráció: a feladatok, dokumentumok és értékelések kezelése a Google Drive, Docs és Forms eszközein keresztül történik.

Hátránya azonban, hogy az intézményi adatok Google szervereire kerülnek, ami adatvédelmi – különösen GDPR-szempontból – problémákat vethet fel az európai intézményeknél [11]. A testreszabhatósága korlátozott: nincs lehetőség saját adatbázis-séma, egyedi jogosultsági logika vagy külső rendszerekkel való mélyebb integráció kialakítására. Önállóan üzemeltetett, offline vagy intranetes környezetben nem működőképes.

2.2.3. Canvas LMS

A Canvas LMS-t az Instructure fejlesztette ki, és 2011-es megjelenése óta az egyik leginnovatívabb kereskedelmi LMS-nek számít [13]. Moderne REST API-alapú architektúrája, reszponzív felülete és kiterjedt integrációs lehetőségei (LTI-kompatibilitás, OAuth2 alapú SSO) miatt különösen népszerű az angolszász felsőoktatásban.

A Canvas nyílt forráskódú verziója (Canvas Open Source) elérhető GitHub-on, ugyanakkor a saját infrastruktúrán való telepítése és üzemeltetése jelentős erőforrást és mélyebb DevOps-ismereteket igényel (Docker, Ruby on Rails, PostgreSQL). A licenszköteles, felhőalapú verzió árképzése nagy intézmények számára tervezte, kisebb szervezetek számára nem gazdaságos.

2.2.4. Open edX

Az Open edX a Harvard Egyetem és az MIT által alapított edX platform nyílt forráskódú változata, amelyet a Axim Collaborative tart karban [4]. A platform különlegessége, hogy MOOC-ok (Massive Open Online Courses) futtatására is alkalmas: skálázható, mikroszolgáltatás-alapú architektúrával rendelkezik. Python (Django) backenddel és React frontenddel készült. Hátránya, hogy telepítése és üzemeltetése rendkívül komplex, és kisebb, néhány száz felhasználóra tervezett rendszer esetén indokolatlanul nagy infrastruktúrát igényel.

2.2.5. Összehasonlítás

A fenti rendszerek áttekintése alapján megállapítható, hogy mindegyikük valamilyen kompromisszumot kényszerít az üzemeltetőre: vagy a komplexitás és erőforrásigény, vagy az adatfeletti kontroll hiánya, vagy a testreszabhatóság korlátai jelentenek akadályt. A ME Learning ezt a rést tölti be azáltal, hogy egyszerűen telepíthető, saját szerveren futtatható, Java alapú és nyílt forráskódú.

2.1. táblázat: E-learning rendszerek összehasonlítása

Rendszer	**Nyílt forrás**	**Saját szerver**	**Egyszerű deploy**
Moodle	Igen	Igen	Nem
Google Classroom	Nem	Nem	Igen
Canvas	Részben	Igen	Nem
Open edX	Igen	Igen	Nem
ME Learning	Igen	Igen	Igen

2.3. Felhasznált technológiák

A fejlesztési technológiák kiválasztása során az alábbi szempontok vezéreltek: egységes, Java-alapú ökoszisztéma használata, minimális külső függőség, jól dokumentált és aktívan karbantartott keretrendszerek preferálása, valamint az alkalmazás egyszerű, önálló JAR-fájlként való terjeszthetősége.

2.3.1. Java és a Spring-ökoszisztéma általában

A Java programozási nyelv az egyik legelterjedtebb platformfüggetlen nyelv a vállalati szoftverfejlesztésben. Az erős típusosság, a JVM (Java Virtual Machine) által biztosított hordozhatóság és a kiterjedt keretrendszer-ökoszisztéma teszi vonzóvá vállalati webalkalmazások fejlesztéséhez [8]. A Spring Framework a Java-ökoszisztéma egyik legkiterjedtebb, inverz vezérlésre (IoC – Inversion of Control) és függőség-injektálásra (DI – Dependency Injection) épülő keretrendszere, amelyet Rod Johnson fejlesztett ki 2002-ben az alkalmazásszerver-centrizmus alternatívájaként [14].

2.3.2. Spring Boot

A Spring Boot a Spring Framework fölé épülő, konvención alapuló konfigurációs réteg, amelyet a Pivotal (ma VMware) tett elérhetővé 2014-ben [20]. Legfontosabb újítása az

auto-configuration mechanizmus: a classpath-on talált függőségek alapján automatikusan konfigurálja az alkalmazáskomponenseket, minimalizálva a manuális XML- vagy Java-konfigurációt.

A Spring Boot beágyazott Tomcat, Jetty vagy Undertow szerverre épít, így az alkalmazás egyetlen futtatható JAR-fájlként csomagolható és indítható el a `java -jar` paranccsal, anélkül, hogy külön alkalmazásszerverre lenne szükség. Ez az *opinionated defaults* (határozott alapbeállítások) elve alapján működik: a keretrendszer bevált konfigurációt ajánl fel, amelyet a fejlesztő szükség esetén felülírhat. A ME Learning esetén Spring Boot 3.x verzióját alkalmaztam, amely Java 17-es LTS (Long-Term Support) kiadáson fut, és natívan támogatja a Jakarta EE 10 API-t.

2.3.3. Spring Security

A Spring Security a Spring-ökoszisztéma hitelesítési (authentication) és engedélyezési (authorization) modulja [19]. A rendszerben a következő három szerepkör kerül definiálásra:

- `ROLE_ADMIN` – teljes körű rendszerfelügyelet, felhasználókezelés;
- `ROLE_INSTRUCTOR` – kurzusok és kvízek kezelése, hallgatók menedzselése;
- `ROLE_STUDENT` – kurzusok böngészése, beiratkozás, kvízkitöltés.

A Spring Security filter chain mechanizmusa alapján minden HTTP-kérés átmegy egy biztonsági szűrőláncon, amely ellenőrzi a hitelesítési tokent (session-alapú, cookieból olvasott JSESSIONID), majd a `@PreAuthorize` annotációk vagy `HttpSecurity` konfiguráció alapján eldönti, hogy a kérés jogosult-e az adott erőforráshoz hozzáférni.

A jelszavak tárolása BCrypt algoritmust alkalmaz. A BCrypt egy adaptív hashfüggvény, amelyet Niels Provos és David Mazières dolgozott ki 1999-ben [16]. Fő előnye, hogy a *work factor* (munkatényező) paraméterezhető: ez lehetővé teszi, hogy a számítási teljesítmény növekedésével arányosan növeljük a jelszó-feltörési kísérletek erőforrásigényét, ezáltal megőrizve a jelszavak biztonságát.

2.3.4. Spring Data JPA és Hibernate

Az adatbázis-hozzáférés JPA (Java Persistence API) szabványon keresztül valósul meg. A JPA egy Java EE (ma Jakarta EE) specifikáció, amely az objektum-relációs leképezés (ORM – Object-Relational Mapping) egységes API-ját írja le [5]. Az alkalmazásban a referenciaimplementációt a Hibernate ORM biztosítja, amely az egyik legelterjedtebb és leghosszabb múltra visszatekintő Java ORM-keretrendszer.

A Hibernate az entitásosztályokat (`@Entity` annotációval jelölt Java POJO-k) automatikusan leképezi adatbázistáblákra, és a JPQL (Java Persistence Query Language) vagy natív SQL lekérdezéseken keresztül lehetővé teszi az adatok hatékony kezelését. A `spring.jpa.hibernate.ddl-auto=update` konfiguráció fejlesztési fázisban gondoskodik arról, hogy a séma automatikusan frissüljön az entitásváltozások alapján.

A fejlesztési fázisban H2 beágyazott, memóriában futó adatbázist alkalmazunk, amely nem igényel külön telepítést, és az alkalmazás indításakor inicializálódik. Éles környezetben ez könnyedén felváltható PostgreSQL vagy MySQL adatbázisra az `application.properties` fájlban megadott JDBC-kapcsolati paraméterek módosításával – az alkalmazáskód változtatása nélkül.

2.3.5. Thymeleaf

A Thymeleaf egy modern, szerver oldali Java sablon-motor, amelyet Daniel Fernández fejlesztett ki [9]. Fő jellemzője, hogy *természetes sablonokat* (natural templates) alkalmaz: a HTML-fájlok Thymeleaf-attribútumokkal (pl. `th:text`, `th:each`, `th:if`) egészülnek ki, de statikusan, böngészőben is megnyithatók maradnak, ami megkönnyíti a frontend-fejlesztők munkáját.

A Thymeleaf és a Spring Security integrációja lehetővé teszi, hogy a nézetekben a felhasználó aktuális szerepköre alapján dinamikusan jelenítsük meg vagy rejtsük el az egyes HTML-elemeket. Erre a `sec:authorize` attribútum-névtér szolgál: például `sec:authorize="hasRole('ADMIN')"` esetén az adott blokk csak adminisztrátoroknak látható. Ez deklaratív, HTML-szintű hozzáférés-vezérlést tesz lehetővé anélkül, hogy a nézetlogikában Java-kódot kellene elhelyezni.

2.3.6. Apache POI

Az Apache POI a Microsoft Office dokumentumformátumok Java-alapú olvasásához és írásához használt nyílt forráskódú könyvtár, amelyet az Apache Software Foundation tart karban [3]. A projektben a PowerPoint-fájlok (`.ppt` és `.pptx`) feldolgozásához alkalmazzuk. A `.pptx` formátum valójában OOXML (Office Open XML) alapú ZIP-archívum, amelynek belső struktúráját az Apache POI `XMLSlideShow` osztálya teszi hozzáférhetővé.

A feldolgozás menete a következő: az Apache POI segítségével a prezentáció minden egyes diáját egy `BufferedImage` objektumba rendereljük, amelyet aztán PNG-képként mentünk a szerverre. Ezeket a képeket a Thymeleaf-sablon egy lapozható galériaként jeleníti meg a böngészőben, így a felhasználónak nincs szüksége Microsoft Office-szoftverre a diák megtekintéséhez.

2.3.7. Apache PDFBox

Az Apache PDFBox szintén az Apache Software Foundation projektje, amellyel PDF-dokumentumokat lehet Java-ban programozottan létrehozni, módosítani és feldolgozni [2]. A ME Learningben a feltöltött PDF-prezentációs anyagok diánkénti képrenderelésére alkalmazzuk. A PDFBox `PDFRenderer` osztálya a PDF egyes oldalait konfigurálható felbontású (DPI) képekként jeleníti meg, amelyek ugyanolyan módon tárolódnak és kerülnek megjelenítésre, mint a PowerPoint-diákból generált képek.

Ez az egységes, képalapú megjelenítési megközelítés biztosítja, hogy a tartalom kinézete minden böngészőben azonos legyen, és ne függjön a kliens oldali szoftverkörnyezettől.

2.4. Követelmény-specifikáció

A szoftverkövetelmények összegyűjtése és dokumentálása a szoftverfejlesztési folyamat egyik alapvető lépése. A követelménytervezés célja, hogy egyértelműen rögzítse, mit kell a rendszernek teljesítenie (funkcionális követelmények), és milyen minőségi elvárásoknak kell megfelelnie (nemfunkcionális követelmények) [18]. Az alábbiakban a ME Learning rendszer követelményeit az IEEE 830-as szabvány szellemében mutatom be.

2.4.1. Funkcionális követelmények

A funkcionális követelmények a rendszer konkrét, megfigyelhető viselkedését írják le. Az alábbi lista a fejlesztés során azonosított és implementált funkcionális követelményeket tartalmazza:

- **FR-01 – Felhasználói regisztráció és hitelesítés:** A rendszer legyen képes felhasználók regisztrációjára e-mail cím és jelszó megadásával, valamint biztonságos bejelentkezésre munkamenet-kezeléssel.
- **FR-02 – Szerepkör-alapú hozzáférés-vezérlés:** A rendszer három szerepkört különböztessen meg (`ADMIN`, `INSTRUCTOR`, `STUDENT`), és az egyes funkciókhoz való hozzáférést ezek alapján korlátozza.
- **FR-03 – Kurzuskezelés:** Az oktatók hozhassanak létre, módosíthassanak és törölhessenek kurzusokat, megadva a kurzus nevét, leírását és láthatóságát.
- **FR-04 – Prezentációfeltöltés:** Kurzusokhoz PDF és PowerPoint (`.pptx`) prezentációs fájlok tölthetők fel; a rendszer ezeket böngészőben megtekinthető képekké konvertálja.
- **FR-05 – Kvízkezelés:** Az oktatók kvízeket hozhatnak létre kérdésekkel és helyes válaszokkal; a rendszer automatikusan értékelje a beküldött kitöltéseket.
- **FR-06 – Beiratkozás:** A hallgatók böngészhetik a nyilvános kurzusokat és beiratkozhatnak; az oktatók láthatják a beiratkozott hallgatók listáját.
- **FR-07 – Eredményvisszajelzés:** A kvízkitöltés után a hallgató azonnali visszajelzést kapjon elért eredményéről (helyes/helytelen válaszok, összpontszám).
- **FR-08 – Felhasználókezelés:** Az adminisztrátor megtekinthesse, módosíthassa és törölhesse a felhasználói fiókokat.
- **FR-09 – E-mail értesítés:** A rendszer e-mail értesítést küldjön sikeres regisztráció esetén.

2.4.2. Nemfunkcionális követelmények

A nemfunkcionális követelmények a rendszer minőségi jellemzőit írják le, amelyek nem közvetlenül az üzleti logikára, hanem a rendszer viselkedésének módjára vonatkoznak [6]:

- **NFR-01 – Reszponzivitás:** Az alkalmazás reszponzív, mobilbarát felülettel rendelkezzen, Bootstrap keretrendszer segítségével, hogy különböző képernyőméreteken optimálisan jelenjen meg.
- **NFR-02 – Biztonság:** A jelszavak titkosítva tárolódjanak BCrypt hash-algoritmussal; az alkalmazás legyen védett CSRF (Cross-Site Request Forgery) támadások ellen.
- **NFR-03 – Fájlméret-korlát:** A fájlfeltöltés maximális mérete 50 MB legyen feltöltésenként.

- **NFR-04 – Platformkompatibilitás:** Az alkalmazás Spring Boot 3.x és Java 17 LTS környezeten fusson, és Windows, Linux és macOS operációs rendszereken egyaránt telepíthető legyen.
- **NFR-05 – Karbantarthatóság:** A forráskód rétegelt architektúrát kövessen (Controller–Service–Repository), és a komponensek legyenek egységtesztekkel lefedhetők.
- **NFR-06 – Telepíthetőség:** Az alkalmazás egyetlen futtatható JAR-fájlként legyen terjeszthető, amelynek elindításához csupán egy Java 17 JRE szükséges.

3. fejezet

Tervezés

Ez a fejezet az alkalmazás tervezési döntéseit mutatja be: az architektúrától az adatmodellen és a biztonsági megfontolásokon át a felhasználói felület logikájáig. A tervezési szakasz célja, hogy a megvalósítás előtt strukturált, konzisztens alapot fektessen le, csökkentve a fejlesztés közbeni átdolgozások szükségességét.

3.1. Rendszerarchitektúra

3.1.1. Háromrétegű architektúra

Az alkalmazás klasszikus háromrétegű (three-tier) architektúrát követ, amelyet a Spring MVC keretrendszer természetes módon kényszerít ki:

- **Megjelenítési réteg (Presentation Layer):** Thymeleaf HTML sablonok, CSS stíluslapok és a böngésző oldali megjelenítés. Ez a réteg felelős a felhasználói interakciók fogadásáért és az adatok vizuális megjelenítéséért.
- **Üzleti logika réteg (Business Layer):** Spring MVC kontrollerek és service osztályok. A kontrollerek a HTTP kérések fogadásáért és az eredmények visszaküldéséért felelnek, míg a service osztályok az alkalmazás üzleti szabályait valósítják meg.
- **Adatelérési réteg (Data Access Layer):** Spring Data JPA repository interfészek és a H2 adatbázis. Ez a réteg absztrahálja az adatbázis-műveleteket, lehetővé téve, hogy az üzleti logika réteg ne függjön közvetlenül az adatbázis típusától.

A rétegek közötti kommunikáció szigorúan egyirányú: a kontroller hívja a service-t, a service hívja a repository-t, a repository pedig az adatbázissal kommunikál. Ez a felépítés megfelel a *Separation of Concerns* elvének, és megkönnyíti az egyes rétegek önálló tesztelhetőségét, cserélhetőségét.

3.1.2. Kérés-feldolgozás folyamata

Egy tipikus HTTP kérés feldolgozása az alábbi lépéseken megy keresztül:

1. A böngésző HTTP kérést küld a szervernek (pl. `GET /courses`).

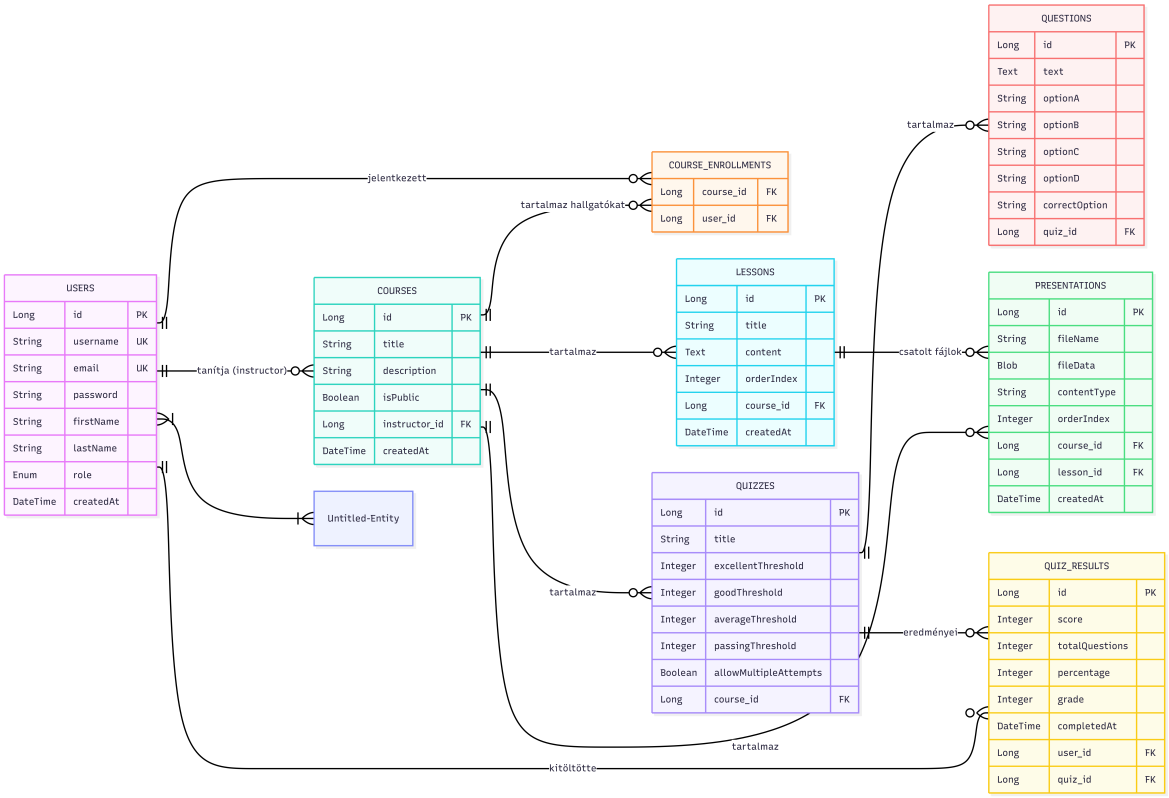
2. A Spring Security `SecurityFilterChain` ellenőrzi, hogy a kérés hitelesített-e, és a felhasználó rendelkezik-e a szükséges jogosultsággal.
3. A `DispatcherServlet` az URL és a HTTP metódus alapján meghatározza a megfelelő kontrollert és metódust.
4. A kontroller meghívja a szükséges service metódusokat az üzleti logika végrehajtásához.
5. A service a repository interfészeken keresztül lekéri vagy módosítja az adatbázisban tárolt adatokat.
6. A kontroller az eredményt egy `Model` objektumba helyezi, és visszaadja a nézet nevét.
7. A Thymeleaf sablon-motor a modell adatai alapján legenerálja a HTML választ.
8. A böngésző megjeleníti a visszakapott HTML oldalt.

3.1.3. Telepítési modell

Az alkalmazás egyetlen önálló JAR-fájlként terjeszthető, amely magában foglalja a beágyazott Tomcat alkalmazásszervert. Ez lényegesen egyszerűsíti a telepítést: nincs szükség különálló alkalmazászerverr konfigurálására. A fejlesztési fázisban beágyazott H2 memória-adatbázis kerül alkalmazásra, amelyet éles környezetben PostgreSQL vagy MySQL adatbázisra lehet felváltani mindössze az `application.properties` módosításával, a Java forráskód megváltoztatása nélkül. Ez a rugalmasság a Spring Data JPA adatbázis-agnosztikus absztrakciójából következik.

3.2. Adatmodell

Az alkalmazás adatmodellje nyolc entitást tartalmaz, amelyek JPA annotációkkal vannak leképezve relációs táblákra. Az entitások tervezésekor a normalizált adatszerkezet és az egyszerű lekérdezhetőség szempontjai egyaránt érvényesültek.



3.1. ábra: Az adatbázis logikai sémája

3.2.1. User

A `User` entitás tárolja a felhasználói adatokat: egyedi felhasználónév (`username`), e-mail cím (`email`), titkosított jelszó (`password`), keresztnév (`firstName`), vezetéknév (`lastName`), szerepkör (`role`: `ADMIN`, `INSTRUCTOR` vagy `STUDENT`) és a regisztráció dátuma (`createdAt`). Egy felhasználó több kurzusra is beiratkozhat, amelyet `@ManyToMany` kapcsolat valósít meg a `Course` entitással, egy köztes `user_courses` kapcsolótáblán keresztül.

A tervezés során tudatos döntés volt, hogy a szerepkör nem külön táblában, hanem egyetlen enum mezőben tárolódik. Ez az egyszerűsítés az alkalmazás léptékéhez igazodik: három, előre rögzített szerepkörrel egy JOIN-műveletet megtakarít minden lekérdezésnél, anélkül hogy a kibővíthetőséget lényegesen korlátozná.

3.2.2. Course

A `Course` entitás képviseli a kurzusokat. Tartalmaz címet (`title`), leírást (`description`), nyilvánossági jelzőt (`isPublic`), az oktató referenciáját (`instructor`, `@ManyToOne` kapcsolat a `User` entitással), leckék listáját (`lessons`, `@OneToMany`), kvízek listáját (`quizzes`, `@OneToMany`), prezentációk listáját (`presentations`, `@OneToMany`), valamint a beiratkozott hallgatók halmazát (`enrolledStudents`, `@ManyToMany`).

Az `isPublic` jelző lehetővé teszi, hogy egyes kurzusok szabadon böngészhetők és csatlakozhatók legyenek, míg más kurzusokba csak az oktató explicit engedélyével lehet belépni. Ez a kétszintű láthatósági modell rugalmasságot biztosít az oktatóknak.

3.2.3. Lesson

A `Lesson` egy kurzuson belüli tematikus leckét jelöl. Tartalmaz egy sorszámot (`orderIndex`), amely meghatározza a leckék megjelenítési sorrendjét a kurzuson belül, szöveges tartalmat (`content`, adatbázisban `TEXT` típusú oszlopban tárolva), a lecke létrehozásának időpontját (`createdAt`), valamint hivatkozást a szülő kurzusra (`course`, `@ManyToOne` kapcsolat a `Course` entitással). Emellett a lecke közvetlen kapcsolatban áll a hozzá feltöltött prezentációkkal is (`presentations`, `@OneToMany` kapcsolat a `Presentation` entitással, `CascadeType.ALL` beállítással). Ez utóbbi kapcsolat teszi lehetővé a lecke-szintű anyagszervezést: az oktató a prezentációkat nem csupán kurzusszinten, hanem konkrét leckékhez rendelve is feltöltheti, így a hallgatók a kurzus megtekintésekor tematikusan, leckénként is böngészhetnek a feltöltött anyagok között. A `Presentation` entitás `lesson` mezője opcionális (lehet `null`), ami azt jelenti, hogy a korábbi, leckéhez nem rendelt kurzusszintű prezentációk is érvényes állapotban maradnak. A sorrend külön mezőben való tárolása lehetővé teszi, hogy az oktatók tetszőlegesen rendezhessék a leckéket az adatbázis-sorok fizikai sorrendjének megváltoztatása nélkül. A leckék lekérdezése mindig az `orderIndex` mező szerint növekvő sorrendben történik.

3.2.4. Presentation

A `Presentation` a feltöltött prezentációs anyagot reprezentálja. Tárolja az eredeti fájlnevet (`originalFilename`), a fájl bináris adatait `byte[]` tömbként (`fileData`), a fájl MIME-típusát (`contentType`), valamint a szülő kurzus referenciáját (`course`, `@ManyToOne`). A bináris tartalom adatbázisban való tárolása egyszerűsíti a telepítést, hiszen nincs szükség külön fájlrendszer-kezelésre; ugyanakkor éles, nagy forgalmú környezetben ez a megközelítés felváltható objektumtárolóval (pl. Amazon S3).

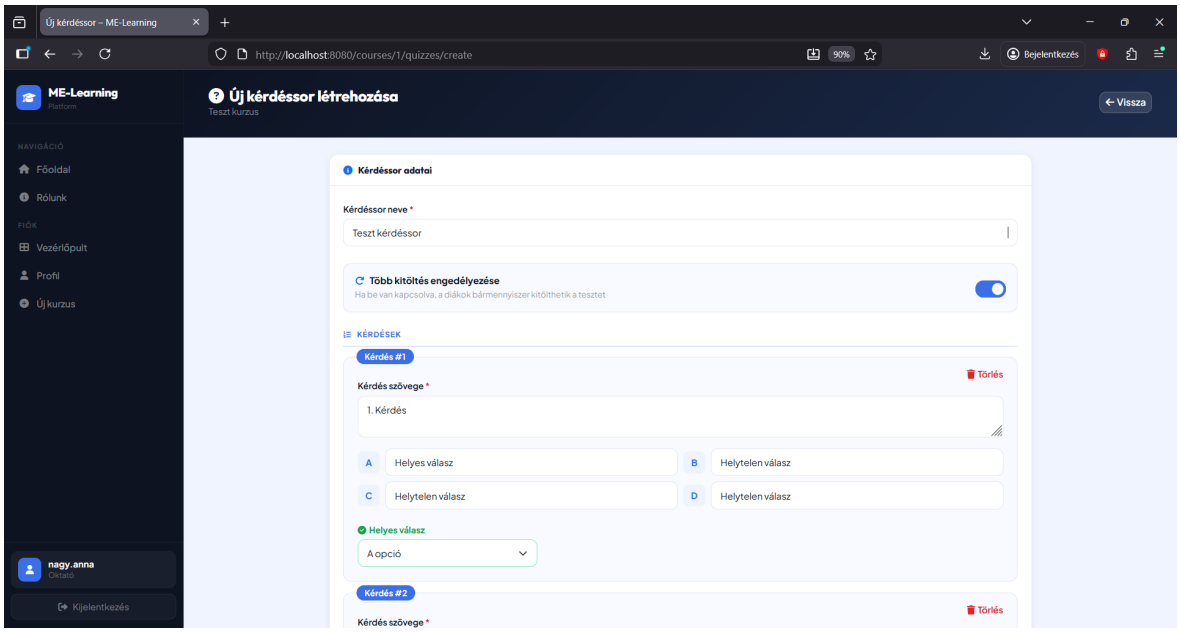
3.2.5. Quiz és Question

A `Quiz` egy kurzushoz rendelt tesztet jelöl (`@ManyToOne` kapcsolat a `Course`-szal). Tartalmazza a kvíz nevét (`title`), azt, hogy engedélyezett-e többszöri kitöltés (`allowMultipleAttempts`), a ponthatárokat (pl. `thresholdExcellent`, `thresholdGood` stb.), valamint a kérdések listáját (`questions`, `@OneToMany`).

A `Question` entitás egyedi kérdést tárol (`questionText`), a lehetséges válaszlehetőségeket (`options`, `List<String>`, amelyet JPA `@ElementCollection` segítségével külön táblában tárol), és a helyes válasz nullaalapú indexét (`correctAnswerIndex`). Az `@ElementCollection` használatával a válaszlehetőségek száma kérdésenként rugalmasan változhat, ami az adatmodell módosítása nélkül is jelentősen megkönnyíti a jövőbeli, eltérő struktúrájú kérdéstípusok implementálását.

3.2.6. QuizResult

A `QuizResult` rögzíti egy hallgató kvízkitöltésének eredményét: hivatkozást a kitöltő felhasználóra (`user`, `@ManyToOne`), hivatkozást a kvízre (`quiz`, `@ManyToOne`), a szerzett pontszámot (`score`), a maximálisan elérhető pontszámot (`maxScore`), a kitöltés időpontját (`completedAt`) és a generált osztályzatot (`grade`). Az osztályzat a ponthatárok alapján kerül kiszámításra, és a kvíz ponthatárainak utólagos módosításakor automatikusan újraszámításra kerül.



3.2. ábra: Kérdéssor létrehozása

3.2.7. Role

A `Role` egy Java `enum` típus, amely az `ADMIN`, `INSTRUCTOR` és `STUDENT` értékeket veheti fel. JPA szinten `@Enumerated(EnumType.STRING)` annotációval tárolódik, ami olvashatóbb adatbázis-tartalmat eredményez, és megvéd az enum-sorrend megváltozásából eredő hibáktól.

3.2.8. Entitáskapcsolatok összefoglalása

3.1. táblázat: Az adatmodell entitásai és főbb kapcsolataik

Forrás entitás	Kapcsolat típusa	Cél entitás	Szemantika
User	ManyToMany	Course	Hallgatói beiratkozás
Course	ManyToOne	User	Oktató tulajdonlás
Course	OneToMany	Lesson	Leckék listája
Course	OneToMany	Quiz	Kvízek listája
Course	OneToMany	Presentation	Feltöltött anyagok
Quiz	OneToMany	Question	Kérdések listája
QuizResult	ManyToOne	User	Kitöltő hallgató
QuizResult	ManyToOne	Quiz	Kitöltött kvíz

3.3. Biztonsági tervezés

3.3.1. Szerepkör-alapú hozzáférés-vezérlés

Az alkalmazás szerepkör-alapú hozzáférés-vezérlést (Role-Based Access Control, RBAC) alkalmaz. A három szerepkör hierarchikusan épül egymásra:

- **STUDENT:** A legszélesebb felhasználói csoport. Böngészhetik és megtekinthetik a nyilvános kurzusokat, beiratkozhatnak, megtekinthetik a feltöltött prezentációkat, és kitölthetik a kvízeket. Saját eredményeiket és profiljukat megtekinthetik.
- **INSTRUCTOR:** A STUDENT jogosultságain felül kurzusokat hozhatnak létre, szerkeszthetnek és törölhetnek, prezentációkat tölthetnek fel, kvízeket kezelhetnek, és kezelhetik a saját kurzusaikra beiratkozott hallgatókat. Más oktatók kurzusait nem módosíthatják.
- **ADMIN:** Teljes hozzáféréssel rendelkezik. Minden kurzust kezelhet a tulajdonostól függetlenül, a felhasználói fiókok tekintetében pedig törlési jogosultsággal bír.

3.3.2. URL-szintű hozzáférés-vezérlés

A Spring Security `SecurityFilterChain` konfiguráció URL-minta alapján szabályozza a hozzáférést. A tervezés során az alábbi hozzáférési zónákat határoztuk meg:

3.2. táblázat: URL-zónák és hozzáférési szabályok

URL-minta	Szükséges jogosultság
`/`, `/login`, `/register`, `/about`	Nyilvános (hitelesítés nélkül)
`/courses`, `/courses/{id}/view`	Hitelesített felhasználó
`/courses/create`, `/courses/*/manage`	INSTRUCTOR vagy ADMIN
`/courses/*/quizzes/create`	INSTRUCTOR vagy ADMIN
Minden egyéb	Hitelesített felhasználó

Emellett a service rétegben is érvényesülnek tulajdonosi ellenőrzések: például a kurzus törlése előtt a rendszer ellenőrzi, hogy a bejelentkezett felhasználó a kurzus oktatója-e, vagy adminisztrátor-e. Ez a kétrétegű ellenőrzés védelmet nyújt a közvetlen URL-manipulációs kísérletek ellen is.

3.3.3. Jelszókezelés

A jelszavak tárolása BCrypt algoritmussal történik, amely egy adaptív, szándékosan lassú hash-függvény. A BCrypt automatikusan sózza (salt) a jelszavakat, így azonos jelszavak is különböző hash-értéket kapnak, ami megvéd a rainbow table alapú támadások ellen. A `BCryptPasswordEncoder` alapértelmezett erőssége (cost factor 10) iparági standardnek megfelelő védelmet biztosít.

3.3.4. Munkamenet-kezelés

A Spring Security alapértelmezett munkamenet-kezelését alkalmazzuk: a bejelentkezés után a felhasználó azonosítója HTTP munkamenetben (session cookie-ban) tárolódik. A munkamenet CSRF (Cross-Site Request Forgery) védelemmel egészül ki, amelyet a Spring Security automatikusan aktivál. Minden állapotváltoztató HTTP POST kérésnek tartalmaznia kell a szerver által generált CSRF tokent, amelyet a Thymeleaf sablonok `th:action` attribútuma automatikusan beilleszt.

3.4. Felhasználói felület tervezése

3.4.1. Általános UX-elvek

A felhasználói felület tervezésekor a következő elvek érvényesültek:

- **Szerepkör-tudatos megjelenítés:** A felhasználói felület dinamikusan alkalmazkodik a bejelentkezett felhasználó szerepköréhez. A Thymeleaf `sec:authorize` direktívája segítségével egyes elemek (pl. „Kurzus létrehozása" gomb, szerkesztési ikonok) csak az arra jogosult szerepkörök számára jelennek meg.
- **Visszajelzés és validáció:** Minden form esetén a szerver oldali validáció (Bean Validation / `@Valid`) azonnali visszajelzést ad a felhasználónak a hibás adatokról, és a korábban megadott helyes adatokat megőrzi az újratöltés után.
- **Konzisztens navigáció:** Minden autentikált oldal egységes fejléccel, navigációs sávval és oldalsávval rendelkezik, amelyet az újrafelhasználható `fragments/sidebar.html` Thymeleaf komponens valósít meg.
- **Reszponzív elrendezés:** A CSS stíluslapok mobilbarát elrendezést biztosítanak, amely különböző képernyőméreteken is használható marad.

3.4.2. Oldalak és navigációs folyamatok

Az alkalmazás nézetei három fő területre oszthatók:

Nyilvános terület (hitelesítés nélkül elérhető): A főoldal (`index.html`) és a tájékoztató oldal (`about.html`) bárki számára megtekinthető. A bejelentkezési (`login.html`) és regisztrációs (`register.html`) formok a hitelesítési folyamat belépési pontjai.

Kurzusterület (hitelesített felhasználóknak): A kurzuslista (`courses/list.html`) a szerepkörtől függő szűréssel jeleníti meg az elérhető kurzusokat. A kurzusrészletoldal (`courses/view.html`) tartalmazza a beiratkozás lehetőségét, a leckék listáját, a prezentáció megtekintőt és a kvízek elérését. Az oktató számára a kezelőfelület (`courses/manage.html`) biztosítja a kurzus tartalmának és beiratkozott hallgatóinak menedzselését.

Irányítópultok (szerepkörtől függően): A `DashboardController` szerepkör alapján irányítja a felhasználót a megfelelő irányítópultra. Az admin irányítópult összesített statisztikákat és gyors navigációt kínál. Az oktató irányítópult a saját kurzusokat és azok hallgatóit, kvízeredményeit foglalja össze. A hallgató irányítópult a beiratkozott kurzusokat és a saját kvízeredményeket listázza.

3.4.3. Sablon-hierarchia és töredékek

A nézetek hierarchikus sablon-struktúrát követnek a kódismétlés elkerülése érdekében. Az `fragments/sidebar.html` töredék egy újrafelhasználható oldalsáv-komponenst definiál, amelyet a `th:replace` Thymeleaf direktívával illesztenek be az összes, autentikált területhez tartozó oldalba. Ez biztosítja, hogy az oldalsáv módosítása egyetlen helyen elvégezhető legyen.

A hibakezelés dedikált nézetekkel valósul meg: a `error/404.html` az ismeretlen URL-ekre, az `error/error.html` az általános szerver-hibákra ad testre szabott, informatív visszajelzést.

3.5. Kvízrendszer tervezése

A kvízrendszer tervezésekor kiemelt figyelmet kapott a rugalmasság és a megbízható eredményszámítás. Az alábbiakban a legfontosabb tervezési döntések kerülnek bemutatásra.

3.5.1. Többszöri próbálkozás kezelése

A kvíz szintjén konfigurálható, hogy engedélyezett-e a többszöri kitöltés (`allowMultipleAttempts`). Ha ez le van tiltva, a rendszer a kvíz megkezdése előtt ellenőrzi, hogy a felhasználóhoz tartozik-e már `QuizResult` bejegyzés az adott kvízhez. Amennyiben igen, az előző eredményt jeleníti meg az új kitöltési lehetőség helyett. Ez a logika a service rétegben valósul meg, nem a kontrollerben, így egységtesztelhetővé válik.

3.5.2. Ponthatárok és osztályzatok

A kvízhez öt osztályzati sáv definiálható (kiváló, jó, közepes, elégséges, elégtelen), amelyek százalékos küszöbértékekhez kötöttek. Az oktató a ponthatárokat utólag is módosíthatja; ebben az esetben a `QuizService` automatikusan újraszámítja az összes, adott kvízhez tartozó korábbi eredmény osztályzatát.

Ez a döntés azzal az előnnyel jár, hogy az oktatók a kvíz kiértékelése után is finomíthatják a pontozási rendszert anélkül, hogy a hallgatóknak újra kellene tölteniük a tesztet.

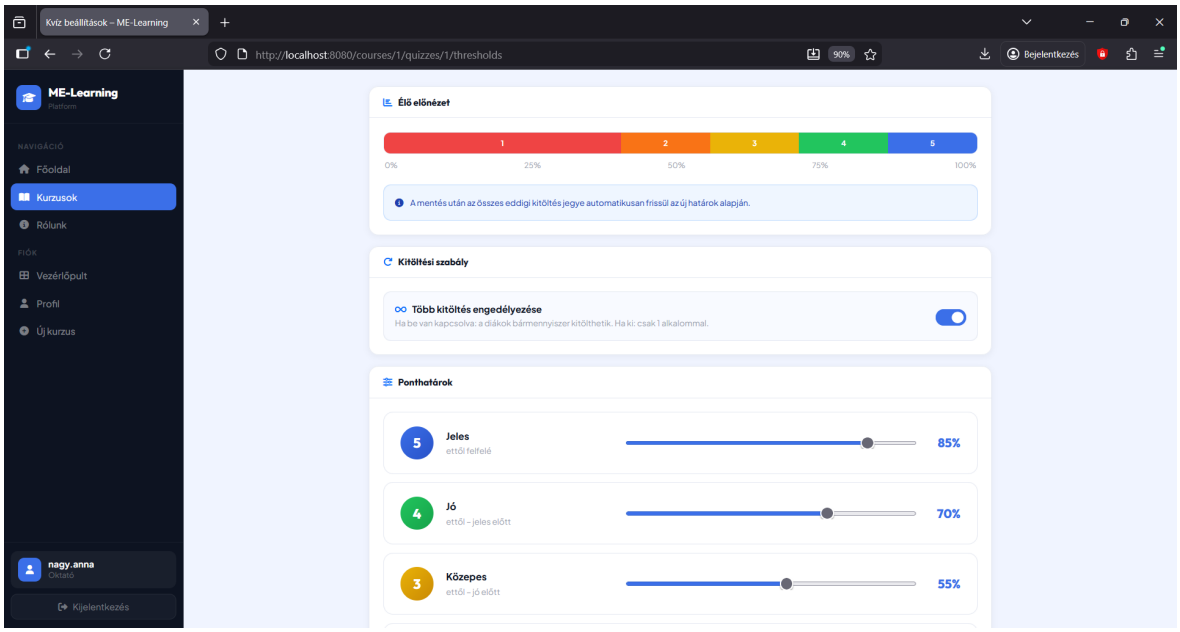
3.5.3. Eredménystatisztikák

Az oktató az összesített eredményoldalon (`quiz-results.html`) a következő mutatókat tekintheti meg: az összes kísérlet száma, az átlagos teljesítmény százalékban, az átlagos osztályzat és az átmenési arány. Ezek a statisztikák a `QuizService`-ben kerülnek kiszámításra a tárolt `QuizResult` rekordok alapján.

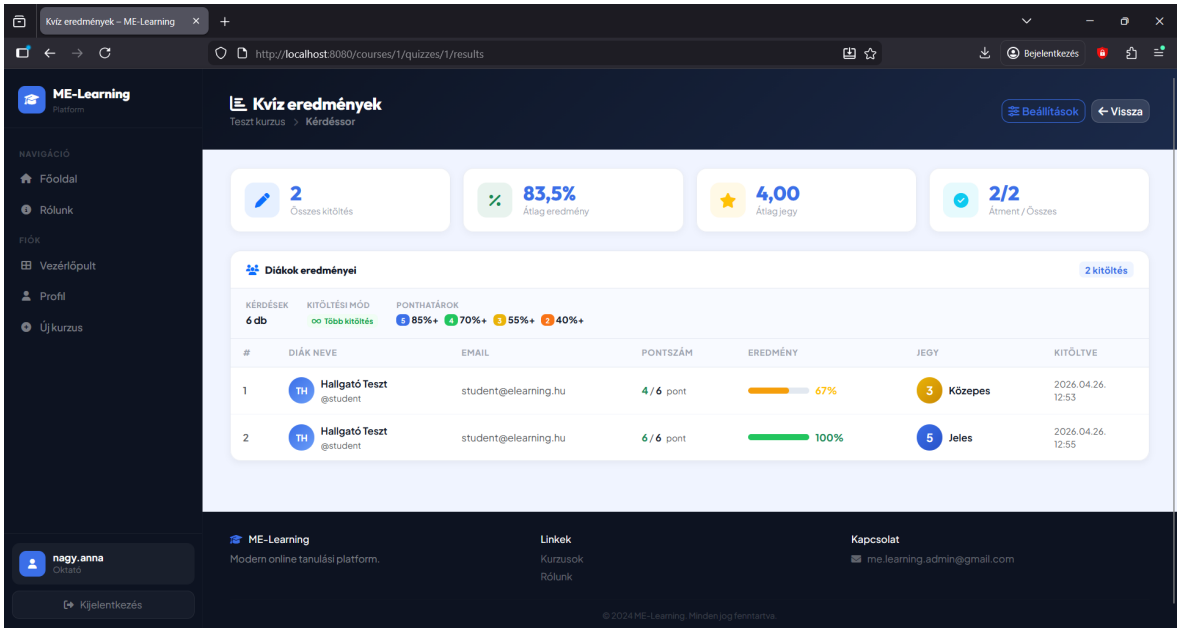
3.6. Prezentáció-feldolgozás tervezése

3.6.1. Támogatott formátumok és konverziós stratégia

Az alkalmazás PowerPoint (`.ppt`, `.pptx`) és PDF fájlok feltöltését támogatja. A tervezési cél az volt, hogy mindkét formátumot egységes módon, böngészőn belüli PDF-



3.3. ábra: Ponthatárok beállítására szolgáló oldal



3.4. ábra: Kvíz kitöltési eredmények megjelenítése

nézetben lehessen megjeleníteni.

PowerPoint fájlok esetén az Apache POI könyvtár segítségével minden dia `BufferedImage`-ként kerül renderelésre 3-szoros skálázási tényezővel, élesebb megjelenés érdekében. Az így kapott képek az Apache PDFBox segítségével egyetlen PDF dokumentummá fűződnek össze, amelyet az adatbázisban tárolunk. PDF fájlok esetén a konverziós lépés kihagyható: a feltöltött PDF közvetlenül tárolódik.

3.6.2. Megjelenítési megközelítés

A tárolt PDF bináris adatokat a `PresentationController` közvetlenül streameli a böngésző felé `application/pdf` MIME-típussal. A böngésző beépített PDF-megjelenítőjét alkalmazzuk, amit egy `<iframe>` elem integrál az oldalba. Ez a megközelítés nem igényel külső JavaScript könyvtárat a megjelenítéshez, és kihasználja a modern böngészők natív PDF-támogatását.

3.6.3. Tárolási megfontolások

A bináris fájladatok (`byte[]`) adatbázisban való tárolása egyszerűsíti a telepítést és az adatkonzisztenciát: nincs szükség szinkronizálásra az adatbázis és a fájlrendszer között. Ugyanakkor ez a megközelítés nagy adatmennyiség esetén növeli az adatbázis méretét. Az architektúra lazán csatolt: a `PresentationService` egy jól definiált interfészt valósít meg, amelynek implementációja a jövőben lecserélhető egy objektumtároló (pl. Amazon S3, MinIO) alapú megoldásra anélkül, hogy a kontroller vagy a nézet réteg változna.

3.7. Réteg-struktúra a forráskódban

Az alkalmazás forráskódja az alábbi csomagokba van szervezve, amelyek pontosan leképezik a háromrétegű architektúrát:

- `config` – A `SecurityConfig` osztály a Spring Security szűrőlánc konfigurációját tartalmazza; a `DataInitializer` az alkalmazás első indítása során alaptesztadatokat hoz létre.
- `controller` – HTTP kéréseket fogadó MVC kontrollerek. Minden fő funkcionális terület saját kontrollert kap: `AuthController` (bejelentkezés, regisztráció), `CourseController` (kurzuskezelés), `DashboardController` (irányítópultok), `HomeController` (főoldal), `PresentationController` (fájl streaming), `QuizController` (kvízkezelés), `UserController` (profilkezelés, adminisztrátori felhasználókezelés).
- `model` – JPA entitásosztályok: `User`, `Course`, `Lesson`, `Presentation`, `Quiz`, `Question`, `QuizResult`, `Role`.
- `repository` – Spring Data JPA interfészek, amelyek az `JpaRepository` alaptípust terjesztik ki. Szükség esetén egyedi lekérdező metódusok (pl. `findByUsername`, `findByInstructor`) kerülnek definiálásra.
- `service` – Üzleti logika osztályok: `CourseService`, `QuizService`, `UserService`, `PresentationService`, `EmailService`, `CustomUserDetailsService`. A `CustomUserDetailsService` implementálja a Spring Security `UserDetailsService` interfészét.

3.7.1. Függőségek iránya

A csomagok közötti függőségek egyirányúak és hierachikusak: a `controller` csomag csak a `service` csomagtól függ, a `service` csomag a `repository` és `model` csomagoktól, a `repository` csomag csak a `model` csomagtól. Ez a struktúra megakadályozza a

körkörös függőségeket, és biztosítja, hogy az egyes rétegek egymástól függetlenül módosíthatók legyenek.

3.8. Nézetek (Thymeleaf sablonok)

A nézetek az alábbi könyvtárakba vannak szervezve, funkcionális területenként:

- `templates/` – Főoldal (`index.html`), bejelentkezés (`login.html`), regisztráció (`register.html`), profil (`profile.html`), saját kurzusok, tájékoztató oldal (`about.html`).
- `templates/courses/` – Kurzuslista (`list.html`), megtekintés (`view.html`), létrehozás (`create.html`), kezelés (`manage.html`), diavetítő nézet (`viewer.html`), kvízkitöltő (`take-quiz.html`), egyéni és összesített eredménynézet (`quiz-result.html`, `quiz-results.html`), ponthatár-szerkesztő (`quiz-thresholds.html`), kvíz-létrehozó (`create-quiz.html`).
- `templates/dashboard/` – Admin (`admin.html`), oktató (`instructor.html`) és hallgató (`student.html`) irányítópultok.
- `templates/presentation/` – Önálló prezentáció megjelenítő (`viewer.html`).
- `templates/fragments/` – Újrafelhasználható oldalsáv komponens (`sidebar.html`).
- `templates/error/` – 404-es hibaoldal (`404.html`) és általános hibaoldal (`error.html`).

A Thymeleaf sablonok természetes HTML dokumentumok, amelyek böngészőben is megnyithatók (statikus prototípusként), és csak a szerver általi feldolgozás során válnak dinamikus tartalmakká. Ez megkönnyíti a nézetek tesztelését és a frontend-fejlesztők munkáját, akiknek nem szükséges a teljes Spring Boot alkalmazást elindítaniuk a megjelenítési munkához.

4. fejezet

Megvalósítás

Ez a fejezet az alkalmazás konkrét implementációját mutatja be, a konfigurációtól az egyes rétegek megvalósításán át a legösszetettebb üzleti logikáig. Minden alfejezetben a tényleges forráskódon keresztül szemléltetjük az adott komponens működését.

4.1. Az alkalmazás indítása és konfiguráció

4.1.1. A belépési pont

A Spring Boot alkalmazás belépési pontja az `ElearningApplication` osztály, amelyet a `@SpringBootApplication` annotáció jelöl. Ez az összetett annotáció három szerepet tölt be egyszerre: engedélyezi az automatikus komponensfelderítést (`@ComponentScan`), aktiválja az auto-configuration mechanizmust (`@EnableAutoConfiguration`), és maga is konfigurációs osztályként viselkedik (`@Configuration`).

Programkód 4.1. Az alkalmazás belépési pontja

```
@SpringBootApplication
public class ElearningApplication {
    public static void main(String[] args) {
        SpringApplication.run(ElearningApplication.class, args);
    }
}
```

A `SpringApplication.run()` hívás indítja el a beágyazott Tomcat szervert, inicializálja az IoC konténert és futtatja az összes `CommandLineRunner` implementációt — köztük az adatinicializálót.

4.1.2. Maven függőségek

A projekt építési leírója a `pom.xml` fájl, amelynek szülőprojektje a `spring-boot-starter-parent` 3.5.6-os verziója. Ez a szülőprojekt egységesen kezeli az összes Spring-függőség verzióját. A főbb függőségek:

- `spring-boot-starter-web` – beágyazott Tomcat és Spring MVC;
- `spring-boot-starter-data-jpa` – Spring Data JPA és Hibernate ORM;
- `spring-boot-starter-security` – Spring Security szűrőlánc;

- `spring-boot-starter-validation` – Bean Validation (Jakarta Validation API);
- `spring-boot-starter-thymeleaf` – Thymeleaf sablon-motor;
- `spring-boot-starter-mail` – JavaMail integráció SMTP-küldéshez;
- `thymeleaf-extras-springsecurity6` – `sec:authorize` névtér a sablonokban;
- `h2` (runtime scope) – beágyazott H2 memória-adatbázis fejlesztési célra;
- `poi-ooxml` és `poi-scratchpad` 5.2.5 – Apache POI `.pptx` és `.ppt` feldolgozáshoz;
- `pdfbox` 2.0.30 – Apache PDFBox PDF-generáláshoz (explicit verziószám szükséges, mert a Spring Boot szülőprojekt nem kezeli);
- `spring-security-test` (test scope) – `@WithMockUser` és CSRF-szimulációs segédeszközök teszteléshez.

4.1.3. Az `application.properties` konfiguráció

A legfontosabb konfigurációs beállítások:

- `spring.datasource.url=jdbc:h2:mem:testdb` – H2 memória-adatbázis;
- `spring.jpa.hibernate.ddl-auto=create-drop` – séma létrehozása indításkor, törlése leálláskor;
- `spring.servlet.multipart.max-file-size=50MB` – fájlfeltöltési méretkorlát;
- `spring.mail.host`, `port`, `username`, `password` – Gmail SMTP konfiguráció;
- `app.mail.from` és `app.base-url` – egyedi property-k az `EmailService` számára;
- `h2.console.security.enabled=false` – H2 konzol letiltása.

4.2. A forráskód szerkezete

A Java forráskód az `src/main/java/com/melearning/elearning/` főcsomagban helyezkedik el, öt alcsomag szerint rendezve:

- `config/` – `SecurityConfig` és `DataInitializer`;
- `controller/` – `AuthController`, `CourseController`, `DashboardController`, `HomeController`, `LessonController`, `PresentationController`, `QuizController`, `UserController`;
- `model/` – `User`, `Course`, `Lesson`, `Presentation`, `Quiz`, `Question`, `QuizResult`, `Role`;
- `repository/` – `CourseRepository`, `LessonRepository`, `PresentationRepository`, `QuestionRepository`, `QuizRepository`, `QuizResultRepository`, `UserRepository`;

- `service/` – `CourseService`, `CustomUserDetailsService`, `EmailService`, `LessonService`, `PresentationService`, `QuizService`, `UserService`.

A csomagok közötti függőségek egyirányúak: controller → service → repository → model.

4.3. Adatmodell megvalósítása

4.3.1. A User entitás

A `User` osztályt a `@Table(name = "users")` annotáció jelöli, mert a `USER` szó fenntartott kulcsszó a H2 SQL dialektusában. Az egyediségi megszorítások adatbázis szinten garantálják, hogy két felhasználó ne regisztrálhasson azonos felhasználónévvel vagy e-mail-címmel.

Programkód 4.2. A User entitás főbb mezői

```
@Entity
@Table(name = "users")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank @Size(max = 50)
    @Column(unique = true)
    private String username;

    @NotBlank @Size(max = 100) @Email
    @Column(unique = true)
    private String email;

    @NotBlank @Size(max = 100)
    private String password;

    @Enumerated(EnumType.STRING)
    private Role role;

    private LocalDateTime createdAt;

    @ManyToMany(mappedBy = "enrolledUsers")
    private Set<Course> enrolledCourses = new HashSet<>();

    public User() { this.createdAt = LocalDateTime.now(); }
    // ... getterek, setterek
}
```

A `@Enumerated(EnumType.STRING)` gondoskodik arról, hogy a `Role` enum értéke szövegként kerüljön az adatbázisba, megvédve az adatokat az enum-sorrend jövőbeli megváltozásától.

4.3.2. A Course entitás

A `Course` entitás a rendszer központi eleme, amely `Lesson`, `Quiz` és beiratkozott `User` entitásokkal is kapcsolatban áll.

Programkód 4.3. A Course entitás kapcsolatai

```
@Entity
@Table(name = "courses")
public class Course {

    @ManyToOne
    @JoinColumn(name = "instructor_id")
    private User instructor;

    @OneToMany(mappedBy = "course",
               cascade = CascadeType.ALL, orphanRemoval = true)
    private List<Lesson> lessons = new ArrayList<>();

    @OneToMany(mappedBy = "course",
               cascade = CascadeType.ALL, orphanRemoval = true)
    private List<Quiz> quizzes = new ArrayList<>();

    @ManyToMany
    @JoinTable(
        name = "course_enrollments",
        joinColumns = @JoinColumn(name = "course_id"),
        inverseJoinColumns = @JoinColumn(name = "user_id")
    )
    private Set<User> enrolledUsers = new HashSet<>();
    // ...
}
```

A `CascadeType.ALL` és `orphanRemoval = true` kombináció a `Lesson` és `Quiz` kapcsolatoknál biztosítja, hogy kurzustörléskor az összes hozzá tartozó lecke és kvíz automatikusan törlődjön.

4.3.3. A Lesson entitás

A `Lesson` entitás egy kurzuson belüli tematikus egységet képvisel. Tartalmaz egy szöveges tartalommezőt (`@Column(columnDefinition = "TEXT")`), egy sorrendi indexet a leckék rendezéséhez, és egy közvetlen kapcsolatot a hozzá csatolt prezentációkhoz.

Programkód 4.4. A Lesson entitás felépítése

```
@Entity
@Table(name = "lessons")
public class Lesson {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
```

```
    @NotBlank @Size(max = 100)
    private String title;

    @Column(columnDefinition = "TEXT")
    private String content;

    private Integer orderIndex;

    @ManyToOne
    @JoinColumn(name = "course_id")
    private Course course;

    @OneToMany(mappedBy = "lesson", cascade = CascadeType.ALL)
    private List<Presentation> presentations = new ArrayList<>()
        ;

    private LocalDateTime createdAt;

    public Lesson() { this.createdAt = LocalDateTime.now(); }
    // ...
}
```

A leckék sorrendbe rendezése az `orderIndex` mező alapján történik, amelyet a `LessonRepository` a `findByCourseIdOrderByOrderIndexAsc()` metódussal vesz figyelembe. A `presentations` lista lehetővé teszi, hogy minden leckéhez közvetlenül csatoljunk feltöltött anyagokat, így a hallgatók a kurzus megtekintésekor lecke-szinten is böngészhetnek a prezentációk között.

4.3.4. A Presentation entitás

Az optimalizálás során a `Presentation` entitás két kapcsolattal rendelkezik: a kötelező `Course`-kapcsolat mellett egy opcionális `Lesson`-kapcsolat is bekerült, `FetchType.LAZY` betöltési stratégiával.

Programkód 4.5. A Presentation entitás bővített kapcsolatai

```
@Entity
@Table(name = "presentations")
public class Presentation {

    @Lob
    @Column(columnDefinition = "LONGBLOB")
    private byte[] fileData;

    @ManyToOne
    @JoinColumn(name = "course_id")
    private Course course;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "lesson_id")
    private Lesson lesson;
```

```
    private Integer orderIndex;
    // ...
}
```

A `lesson` mező `null` értékű lehet: ez olyan prezentációknak felel meg, amelyeket az oktató kurzusszinten töltött fel, nem konkrét leckéhez rendelve. A `FetchType.LAZY` beállítás megakadályozza, hogy a prezentáció listázásakor szükségtelenül betöltődjön a kapcsolódó lecke teljes objektuma.

4.3.5. A Quiz és Question entitás

A `Quiz` osztályban az osztályzatot számító `calculateGrade(int percentage)` metódus az entitásban helyezkedik el, mert az osztályzatszámítás szorosan kötődik a kvíz egyedi ponthatáraihoz.

Programkód 4.6. A Quiz entitás pontszámítási metódusa

```
@Entity
@Table(name = "quizzes")
public class Quiz {

    private Integer excellentThreshold = 85;
    private Integer goodThreshold      = 70;
    private Integer averageThreshold   = 55;
    private Integer passingThreshold   = 40;
    private Boolean allowMultipleAttempts = true;

    @OneToMany(mappedBy = "quiz",
               cascade = CascadeType.ALL,
               orphanRemoval = true,
               fetch = FetchType.EAGER)
    private List<Question> questions = new ArrayList<>();

    public int calculateGrade(int percentage) {
        if (percentage >= excellentThreshold) return 5;
        if (percentage >= goodThreshold)      return 4;
        if (percentage >= averageThreshold)   return 3;
        if (percentage >= passingThreshold)   return 2;
        return 1;
    }
    // ...
}
```

A kérdések kapcsolatánál `FetchType.EAGER` lett beállítva, mert a kvíz megjelenítésekor mindig szükség van az összes kérdésre. A `Question` entitás négy fix válaszlehetőséget (`optionA`–`optionD`) tárol külön mezőkben, a helyes választ pedig egy betűjeles `correctOption` mező jelöli.

4.3.6. A QuizResult entitás

A `QuizResult` a `percentage` és `grade` mezőket tárolt értékekként rögzíti, ami lehetővé teszi az összesített statisztikák hatékony számítását és a visszamenőleges osztályzat-

újraszámítást a ponthatárok módosításakor.

4.4. Adatinicializálás

A `DataInitializer` osztály a `CommandLineRunner` interfészt implementálja; `run()` metódusát a Spring Boot az alkalmazáskontextus teljes inicializálása után automatikusan meghívja. Az `if (userRepository.count() == 0)` feltétel biztosítja, hogy az inicializálás csak üres adatbázis esetén fusson le.

Programkód 4.7. Adatinicializáló — tesztfelhasználók létrehozása

```
@Component
public class DataInitializer implements CommandLineRunner {

    @Autowired private UserRepository   userRepository;
    @Autowired private LessonRepository lessonRepository;
    @Autowired private PasswordEncoder  passwordEncoder;

    @Override
    public void run(String... args) throws Exception {
        if (userRepository.count() == 0) {
            initializeUsers();
        }
    }

    private void initializeUsers() {
        User admin = new User();
        admin.setUsername("admin");
        admin.setEmail("admin@elearning.hu");
        admin.setPassword(passwordEncoder.encode("admin123"));
        admin.setRole(Role.ADMIN);
        userRepository.save(admin);

        User instructor = new User();
        instructor.setUsername("instructor");
        instructor.setEmail("instructor@elearning.hu");
        instructor.setPassword(passwordEncoder.encode("
           instructor123"));
        instructor.setRole(Role.INSTRUCTOR);
        userRepository.save(instructor);
    }

    private void createLesson(Course course, String title,
                              String content, int order) {
        Lesson lesson = new Lesson();
        lesson.setTitle(title);
        lesson.setContent(content);
        lesson.setOrderIndex(order);
        lesson.setCourse(course);
        lessonRepository.save(lesson);
    }
```

```
}
```

Az osztály elsődleges célja a tesztkörnyezet gyors felállítása, biztosítva a fejlesztéshez szükséges alapértelmezett tesztadatok (mock adatok) automatikus betöltését.

4.5. Biztonsági konfiguráció

4.5.1. A SecurityConfig osztály

A `SecurityConfig` osztály az alkalmazás biztonsági infrastruktúrájának központi eleme. Feladata a hitelesítés és a jogosultságkezelés szabályozása, amelyet három alapvető Bean definiálásával valósít meg:

- **PasswordEncoder**: A jelszavak biztonságos, BCrypt algoritmussal történő hash-eléséért felel.
- **AuthenticationManager**: A felhasználók azonosításának folyamatát koordinálja.
- **SecurityFilterChain**: Meghatározza a védelmi szűrőket, valamint az URL-alapú hozzáférési szabályokat.

Programkód 4.8. A SecurityFilterChain konfigurálása

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http)
        throws Exception {

    http.userDetailsService(userDetailsService);

    http.authorizeHttpRequests(authz -> {
        if (h2ConsoleEnabled) {
            authz.requestMatchers("/h2-console/**").hasRole("
               ADMIN");
        }

        authz.requestMatchers(
                "/", "/login", "/register", "/about",
                "/css/**", "/js/**", "/images/**"
        ).permitAll();

        authz.requestMatchers(
                "/courses/create",
                "/courses/*/manage",
                "/courses/*/delete",
                "/courses/*/delete-presentation",
                "/courses/*/add-student",
                "/courses/*/remove-student",
                "/courses/*/add-presentations",
                "/courses/*/quizzes/create",
                "/courses/*/quizzes/*/delete",
```

```
                "/courses/*/quizzes/*/thresholds",
                "/courses/*/quizzes/*/results"
        ).hasAnyRole("INSTRUCTOR", "ADMIN");

        authz.requestMatchers(
                "/dashboard/admin/**"
        ).hasRole("ADMIN");

        authz.anyRequest().authenticated();
    });

    http.formLogin(form -> form
            .loginPage("/login")
            .defaultSuccessUrl("/", true)
            .failureUrl("/login?error=true")
            .permitAll()
    );

    http.logout(logout -> logout
            .logoutSuccessUrl("/")
            .permitAll()
    );

    if (h2ConsoleEnabled) {
        http.csrf(csrf -> csrf
                .ignoringRequestMatchers("/h2-console/**"));
        http.headers(headers -> headers
                .frameOptions(frame -> frame.sameOrigin()));
    }

    return http.build();
}
```

A biztonsági szabályrendszer az alábbi hozzáférési szinteket különíti el:

1. **Nyilvános elérés:** A kezdőoldal, a regisztráció, a statikus erőforrások (CSS, JS), valamint az `/about` tájékoztató oldal bárki számára elérhető.

2. **Oktatói és Admin jogosultság:** A kurzusok kezeléséhez, törléséhez és a kvízek adminisztrációjához szükséges funkciók az `INSTRUCTOR` vagy `ADMIN` szerepkörrel rendelkező felhasználókra korlátozódnak.

3. **Kizárólagos Admin elérés:** A rendszerszintű beállítások, a felhasználói fiókok kezelése és — engedélyezés esetén — a H2-konzol elérése csak az `ADMIN` szerepkör számára megengedett.

4.5.2. A CustomUserDetailsService

A `CustomUserDetailsService` implementálja a Spring Security `UserDetailsService` interfészét. A bejelentkezéskor a `loadUserByUsername()` metódus a `UserRepository`-n keresztül tölti be a felhasználót, és a `CustomUserPrincipal` belső osztályba csomagolja.

Programkód 4.9. CustomUserDetailsService és CustomUserPrincipal

```
@Service
public class CustomUserDetailsService implements
   UserDetailsService {

    @Autowired
    private UserRepository userRepository;

    @Override
    public UserDetails loadUserByUsername(String username)
            throws UsernameNotFoundException {
        User user = userRepository.findByUsername(username)
                .orElseThrow(() -> new UsernameNotFoundException
                   (
                        "Felhasználó nem található: " + username
                           ));
        return new CustomUserPrincipal(user);
    }

    public static class CustomUserPrincipal implements
       UserDetails {
        private final User user;

        public CustomUserPrincipal(User user) { this.user = user
           ; }

        @Override
        public Collection<? extends GrantedAuthority>
           getAuthorities() {
            return Collections.singleton(new
               SimpleGrantedAuthority(
                    "ROLE_" + user.getRole().name()));
        }

        @Override public String getPassword() { return user.
           getPassword(); }
        @Override public String getUsername() { return user.
           getUsername(); }
        @Override public boolean isAccountNonExpired()     {
           return true; }
        @Override public boolean isAccountNonLocked()      {
           return true; }
        @Override public boolean isCredentialsNonExpired() {
           return true; }
        @Override public boolean isEnabled()               {
           return true; }

        public User getUser() { return user; }
    }
}
```

A `getUser()` segédmetódus lehetővé teszi, hogy a kontrollerekben az `Authentication` objektumból közvetlenül elérjük a `User` entitást anélkül, hogy külön adatbázislekérdezést kellene indítani.

4.6. Adatelérési réteg

A Spring Data JPA repository interfészek az `JpaRepository<T, ID>` alaptípust terjesztik ki, az egyedi lekérdezéseket pedig metódusnév-alapú query derivation segítségével valósítják meg.

Programkód 4.10. A CourseRepository és a LessonRepository

```
@Repository
public interface CourseRepository extends JpaRepository<Course,
   Long> {
    List<Course> findByInstructor(User instructor);
    List<Course> findByEnrolledUsersContaining(User user);
    List<Course> findByIsPublicTrue();
}

@Repository
public interface LessonRepository extends JpaRepository<Lesson,
   Long> {
    List<Lesson> findByCourseIdOrderByOrderIndexAsc(Long
       courseId);
}
```

A `LessonRepository` egyetlen egyedi metódusa rendezett lekérdezést végez: a `findByCourseIdOrderByOrderIndexAsc()` az `orderIndex` mező szerint növekvő sorrendben adja vissza a leckéket, ezért nincs szükség Java-oldalon utólagos rendezésre.

Programkód 4.11. A QuizResultRepository egyedi metódusai

```
@Repository
public interface QuizResultRepository
        extends JpaRepository<QuizResult, Long> {

    List<QuizResult> findByUser(User user);
    List<QuizResult> findByQuiz(Quiz quiz);
    boolean existsByUserAndQuiz(User user, Quiz quiz);
    Optional<QuizResult>
       findTopByUserAndQuizOrderByCompletedAtDesc(
            User user, Quiz quiz);
}
```

A `findTopByUserAndQuizOrderByCompletedAtDesc` a megadott felhasználóhoz és kvízhez tartozó legfrissebb eredményt adja vissza egyetlen rekordként, amelyet az egyszeri kitöltési logika használ az előző eredmény megjelenítéséhez.

4.7. Szolgáltatási réteg

4.7.1. UserService

A `UserService` az összes felhasználóhoz kapcsolódó üzleti logikát összefogja: regisztrációt, profilkezelést, jelszóváltoztatást és törlést egyaránt. A `saveUser()` az új felhasználók regisztrációjakor használatos: a jelszót BCrypt-tel enkódolja mentés előtt. A profilszerkesztési végpontok az `updateUser()` metódust hívják meg. A felhasználó jelszavának megváltoztatása a `changePassword()` metóduson keresztül történik, amely először ellenőrzi, hogy a megadott jelenlegi jelszó egyezik-e a tárolt hash-sel. Sikeres ellenőrzés után enkódolja és menti az új jelszót, majd az `EmailService`-en keresztül biztonsági értesítőt küld.

Programkód 4.12. Jelszóváltoztatás logikája a UserService-ben

```
public boolean changePassword(User user,
        String currentPassword,
        String newPassword) {
    if (!passwordEncoder.matches(currentPassword, user.
       getPassword())) {
        return false;
    }
    user.setPassword(passwordEncoder.encode(newPassword));
    userRepository.save(user);
    emailService.sendPasswordChangeEmail(
            user.getEmail(), user.getFirstName());
    return true;
}
```

A `UserController` a profiloldalt két elkülönített `POST` végponton kezeli, amelyek a felhasználó saját adatait módosítják:

- `POST /profile/update-email` — e-mail-cím módosítása. Foglaltság-ellenőrzés után menti az új e-mail-címet, és értesítő e-mailt küld az új címre. Ez a művelet nem igényel újrabelépést.
- `POST /profile/update-password` — jelszó módosítása. Három feltételt ellenőriz egymás után: a jelenlegi jelszó helyességét (`passwordEncoder.matches()`), az új jelszó és a megerősítés egyezőségét, valamint a minimális jelszóhosszt (6 karakter). Sikeres módosítás esetén a munkamenet érvénytelenítésre kerül (`SecurityContextHolder.clearContext()`, `request.getSession().invalidate()`), és a felhasználónak az új jelszóval kell újra bejelentkeznie — enélkül a Spring Security a lejárt munkamenetben szereplő régi hitelesítési adatokkal operálna tovább.

4.7.2. CourseService

A `CourseService` osztály a kurzusokhoz kapcsolódó üzleti logikát fogja össze. Metódusai lefedik a kurzusok teljes életciklusát: létrehozást, módosítást és törlést, valamint a különböző szempontú lekérdezéseket — oktató szerinti szűrést, nyilvános kurzusok listázását és beiratkozott hallgatók alapján való keresést. A beiratkozás kezelése szintén

itt történik: a `enrollUser()` és `unenrollUser()` metódusok gondoskodnak a `course_enrollments` kapcsolótábla karbantartásáról. A kurzustörlés `@Transactional` annotációval védett, és a törlés előtt eltávolítja a kapcsolódó prezentációkat, hogy idegen kulcs megsértése ne okozzon adatbázis-hibát.

Programkód 4.13. Kurzustörlés — Presentation előzetes törlésével

```
@Transactional
public void deleteCourse(Long id) {
    courseRepository.findById(id).ifPresent(course -> {
        presentationRepository.deleteByCourse(course);
    });
    courseRepository.deleteById(id);
}
```

A leckék és kvízek `CascadeType.ALL, orphanRemoval = true` beállításuk révén automatikusan törlődnek a kurzussal együtt.

4.7.3. LessonService

A `LessonService` a lecke életciklusának egyszerű kezelését biztosítja: mentést, kurzusazonosító szerinti rendezett lekérdezést és ID alapú keresést.

Programkód 4.14. A LessonService metódusai

```
@Service
public class LessonService {

    @Autowired
    private LessonRepository lessonRepository;

    public Lesson save(Lesson lesson) {
        return lessonRepository.save(lesson);
    }

    public List<Lesson> findByCourseId(Long courseId) {
        return lessonRepository
                .findByCourseIdOrderByOrderIndexAsc(courseId);
    }

    public Optional<Lesson> findById(Long id) {
        return lessonRepository.findById(id);
    }
}
```

4.7.4. QuizService

A `QuizService.saveQuiz()` metódusa gondoskodik a kétirányú `Quiz`–`Question` JPA kapcsolat helyes beállításáról: minden kérdéshez explicit beállítja a szülő-kvíz referenciát, mert enélkül a `quiz_id` idegenkulcs-mező `NULL` értékkel kerülne mentésre.

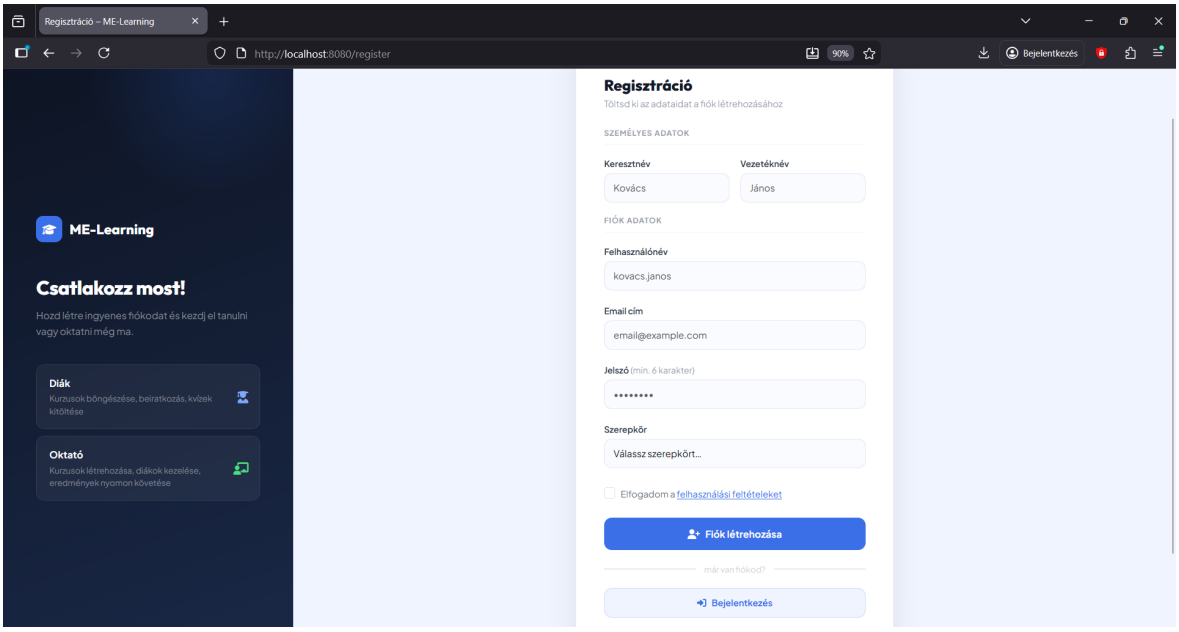
Programkód 4.15. QuizService – kétirányú JPA kapcsolat kezelése mentésnél

```
public Quiz saveQuiz(Quiz quiz) {
    if (quiz.getQuestions() != null) {
        quiz.getQuestions().forEach(q -> q.setQuiz(quiz));
    }
    return quizRepository.save(quiz);
}
```

4.8. Felhasználókezelés megvalósítása

4.8.1. Regisztráció és bejelentkezés

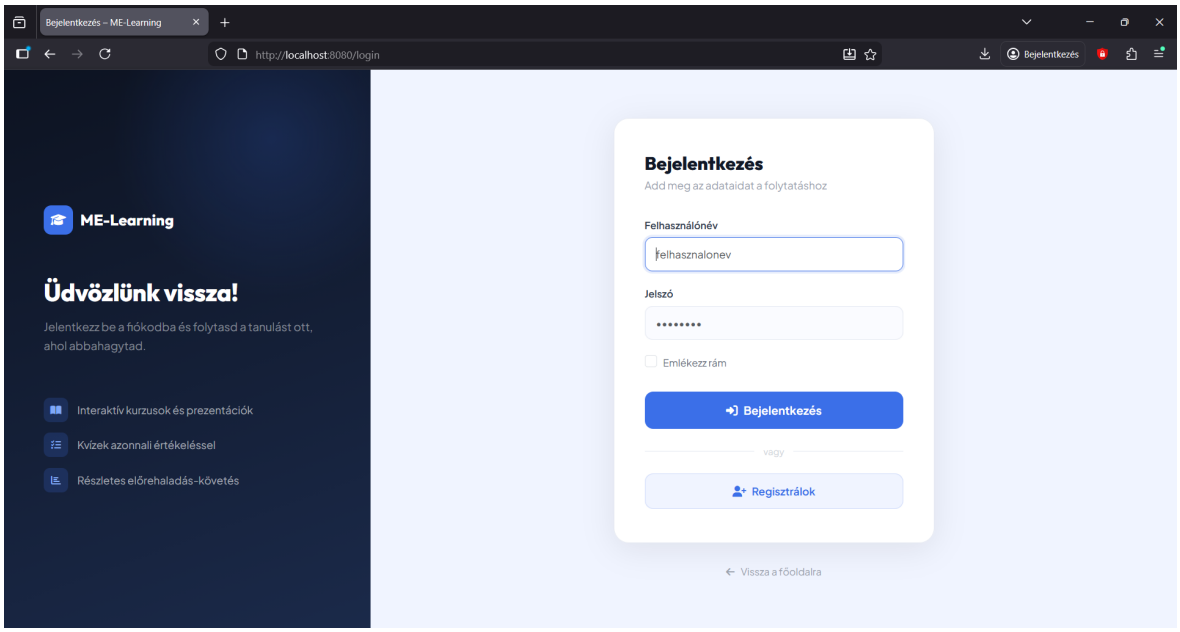
Az `AuthController` kétszintű validációt alkalmaz regisztrációkor: Bean Validation ellenőrzi a mezők formátumát, az alkalmazás üzleti logikája pedig a felhasználónév és e-mail egyediségét.



4.1. ábra: Regisztrációs oldal szemléltetése.

Programkód 4.16. Az AuthController regisztrációs POST végpontja

```
@PostMapping("/register")
public String registerUser(
        @Valid @ModelAttribute("user") User user,
        BindingResult result, Model model,
        RedirectAttributes redirectAttributes) {

    if (result.hasErrors()) return "register";

    if (userService.existsByUsername(user.getUsername())) {
        model.addAttribute("error", "A felhasználónév már
            foglalt!");
        return "register";
```



4.2. ábra: Bejelentkezési oldal szemléltetése.

```
    }
    if (userService.existsByEmail(user.getEmail())) {
        model.addAttribute("error",
                "Az email cím már használatban van!");
        return "register";
    }

    try {
        userService.saveUser(user);
        emailService.sendRegistrationEmail(
                user.getEmail(), user.getFirstName());
        redirectAttributes.addFlashAttribute("success",
                "Sikeres regisztráció! Most már bejelentkezhetsz
                    .");
        return "redirect:/login";
    } catch (Exception e) {
        model.addAttribute("error",
                "Hiba történt a regisztráció során.");
        return "register";
    }
}
```

A bejelentkezési folyamatot maga a Spring Security kezeli a `UsernamePasswordAuthenticationFilter`-en keresztül. Az `AuthController` csak a `GET /login` kérésre adja vissza a nézetet.

4.8.2. Irányítópultok

A `DashboardController` a `GET /dashboard` végponton a felhasználó szerepköre szerint irányít tovább a három dedikált aloldalra (`/dashboard/admin`, `/dash-`

board/instructor, /dashboard/student).

Programkód 4.17. Szerepköralapú irányítópult-átirányítás

```
@GetMapping
public String dashboard(Authentication auth) {
    if (auth == null) return "redirect:/login";
    Optional<User> userOpt =
            userService.getUserByUsername(auth.getName());
    if (userOpt.isEmpty()) return "redirect:/login";

    return switch (userOpt.get().getRole()) {
        case ADMIN      -> "redirect:/dashboard/admin";
        case INSTRUCTOR -> "redirect:/dashboard/instructor";
        case STUDENT    -> "redirect:/dashboard/student";
    };
}
```

Minden aloldal saját `@GetMapping` metódussal rendelkezik, amely a szerepkör-ellenőrzés után betölti az adott szerepkörhöz releváns statisztikákat és listákat. Az admin irányítópult például a felhasználók és kurzusok számát, a hallgatói és oktatói bontást, valamint az összes beiratkozás számát jeleníti meg. Az oktató irányítópultja a saját kurzusait, az ahhoz kapcsolódó hallgatók számát, és a nyilvános/privát kurzusok arányát mutatja. A hallgató irányítópultján a beiratkozott és az elérhető (még nem beiratkozott) nyilvános kurzusok listája jelenik meg.

4.9. Kurzus- és leckekezelés megvalósítása

4.9.1. Kurzuslista keresővel

A `CourseController` a kurzuslista oldalán valós idejű szöveges keresést is támogat az opcionális `keyword` lekérdezési paraméter alapján.

Programkód 4.18. Kurzuslista összeállítása keresési funkcióval

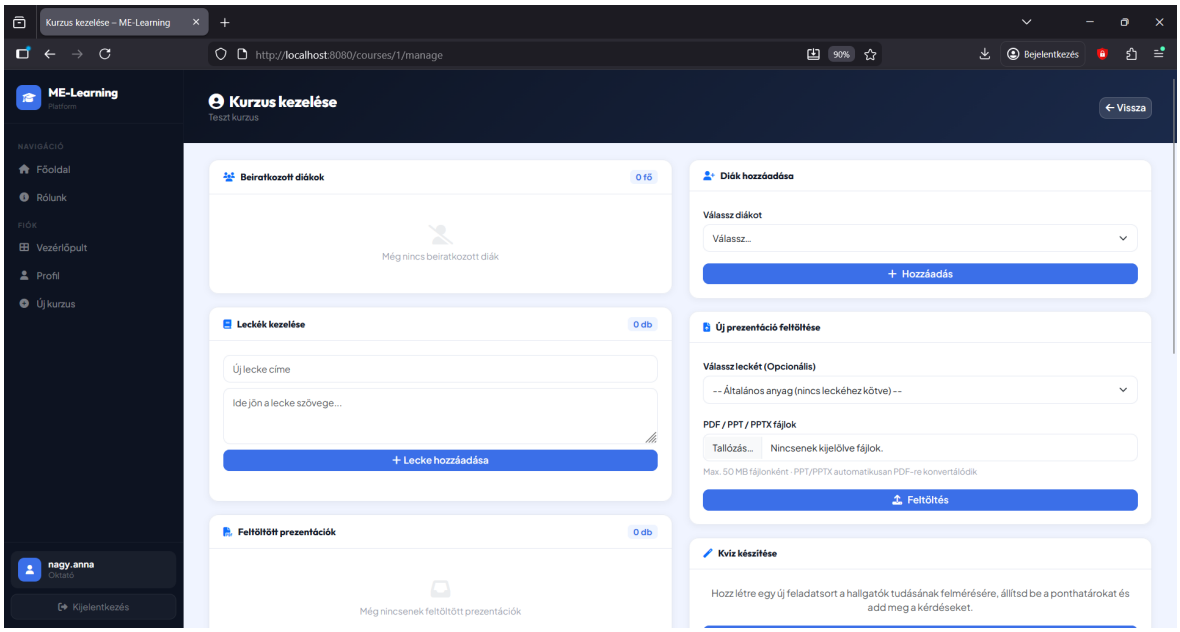
```
@GetMapping
public String listCourses(Model model, Authentication auth,
        @RequestParam(required = false) String keyword) {
    List<Course> courses;
    Optional<User> userOpt = getCurrentUser(auth);

    if (userOpt.isPresent()) {
        User currentUser = userOpt.get();
        if (isAdmin(currentUser)) {
            courses = courseService.getAllCourses();
        } else if (currentUser.getRole() == Role.INSTRUCTOR) {
            courses = courseService.getCoursesByInstructor(
               currentUser);
        } else {
            List<Course> publicCourses =
                    courseService.getPublicCourses();
            List<Course> enrolledCourses =
```

```
                    courseService.getEnrolledCourses(currentUser
                        );
            courses = new ArrayList<>(publicCourses);
            for (Course c : enrolledCourses) {
                if (!courses.contains(c)) courses.add(c);
            }
        }
    } else {
        courses = courseService.getPublicCourses();
    }

    if (keyword != null && !keyword.trim().isEmpty()) {
        String kw = keyword.toLowerCase().trim();
        courses = courses.stream()
                .filter(c -> c.getTitle().toLowerCase().contains
                    (kw)
                    || (c.getDescription() != null
                        && c.getDescription().toLowerCase().
                            contains(kw)))
                .toList();
    }

    model.addAttribute("courses", courses);
    model.addAttribute("keyword", keyword);
    return "courses/list";
}
```

A keresés a szerepkör alapján már szűrt listán fut, tehát egy oktató csak a saját kurzusai között kereshet, a hallgató pedig csak a számára látható kurzusok között.

4.9.2. Kurzus létrehozása

A kurzus létrehozása a `POST /courses/create` végponton keresztül történik. A kontroller először ellenőrzi, hogy a bejelentkezett felhasználó rendelkezik-e a szükséges jogosultsággal, majd beállítja a kurzus oktatóját a bejelentkezett felhasználóra, és elmenti az adatbázisba. Sikeres mentés esetén a felhasználó a kurzuslistára kerül visszairányításra, ahol az újonnan létrehozott kurzus már megjelenik. A kurzus létrehozásakor csak az alapadatok — cím, leírás és láthatóság — kerülnek megadásra; a leckék és prezentációk hozzáadása a létrehozás után, a kurzus kezelőfelületén (`/courses/id/manage`) keresztül történik.



4.3. ábra: Kurzus kezelési oldal

Programkód 4.19. Kurzuslétrehozás

```
@PostMapping("/create")
public String createCourse(
        @Valid @ModelAttribute("course") Course course,
        Authentication auth,
        RedirectAttributes redirectAttributes) {

    Optional<User> instructorOpt = getCurrentUser(auth);

    try {
        course.setInstructor(instructorOpt.get());
        courseService.saveCourse(course);
        redirectAttributes.addFlashAttribute("success",
                "Kurzus sikeresen létrehozva!");
        return "redirect:/courses";
    } catch (Exception e) {
        redirectAttributes.addFlashAttribute("error",
                "Hiba történt a kurzus mentése során: "
                + e.getMessage());
        return "redirect:/courses/create";
    }
}
```

4.9.3. Lecke létrehozása

A lecke létrehozása a `POST /lessons/create` végponton keresztül történik. A kontroller a kérésben érkező `courseId` paraméter alapján azonosítja a szülő kurzust, beállítja a kapcsolatot a lecke és a kurzus között, majd elmenti az adatbázisba. A lecke létrehozásakor az oktató megadja a lecke címét, szöveges tartalmát és létrehozási sorrendben

az indexét (`orderIndex`), amely meghatározza, hogy a lecke milyen sorrendben jelenik meg a kurzus oldalán. Sikeres mentés után a kontroller visszairányítja az oktatót a kurzus kezelőfelületére, ahol az újonnan létrehozott lecke azonnal megjelenik, és prezentációk is csatolhatók hozzá.

Programkód 4.20. LessonController — lecke létrehozása

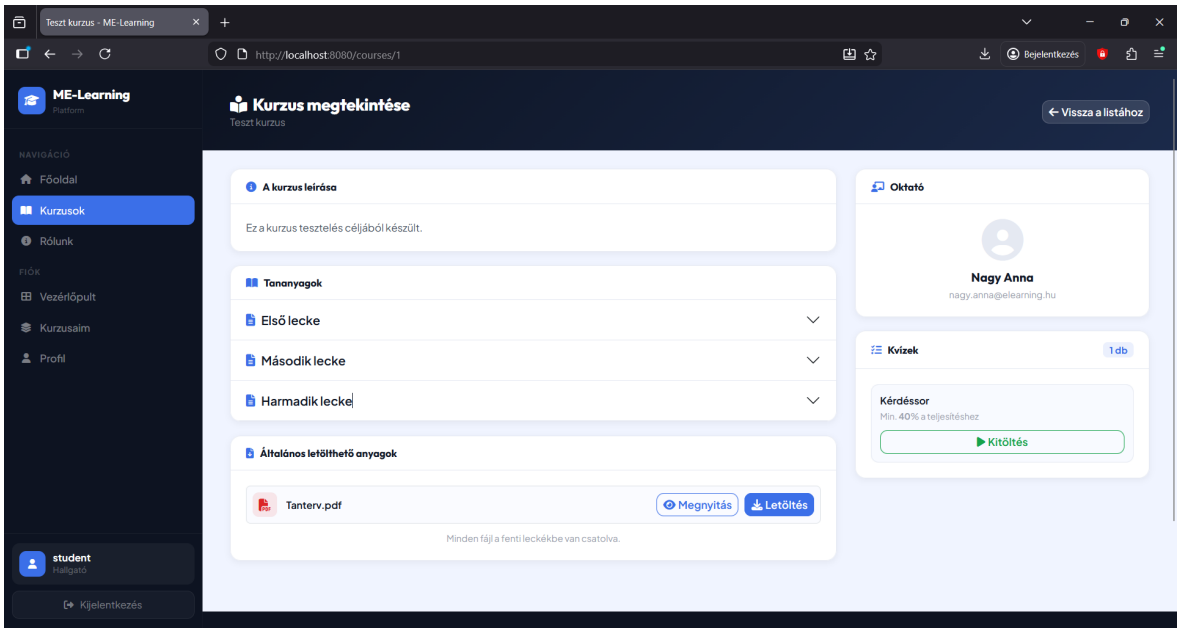
```
@Controller
@RequestMapping("/lessons")
public class LessonController {

    @Autowired private LessonService  lessonService;
    @Autowired private CourseService  courseService;

    @PostMapping("/create")
    public String createLesson(@RequestParam Long courseId,
                               @ModelAttribute Lesson lesson) {
        Course course = courseService.getCourseById(courseId)
                .orElseThrow(() -> new IllegalArgumentException(
                        "Érvénytelen kurzus azonosító: " +
                           courseId));
        lesson.setCourse(course);
        lessonService.save(lesson);
        return "redirect:/courses/" + courseId + "/manage";
    }
}
```

4.9.4. Kurzusnézet leckékkel

A `CourseController` kurzus-megtekintési végpontja a leckéket is betölti a modellbe, hogy a hallgatók a kurzus oldalán lecke-szinten is böngészhessenek.

Programkód 4.21. Kurzusnézet — leckék betöltése a modellbe

```
@GetMapping("/{id}")
public String viewCourse(@PathVariable Long id,
                         Model model, Authentication auth) {
    // ... jogosultság- és átirányítás-logika ...

    model.addAttribute("course",    courseObj);
    model.addAttribute("presentations",
            presentationService.getPresentationsByCourse(
               courseObj));
    model.addAttribute("quizzes",
            quizService.getQuizzesByCourse(courseObj));
    model.addAttribute("lessons",
            lessonService.findByCourseId(courseObj.getId()));

    return "courses/view";
}
```



4.4. ábra: Diákok nézete a beíratkozott kurzusra

A kurzusnézeten az admin és a saját kurzusát megtekintő oktató automatikusan a kezelőfelületre (`manage`) kerül átirányításra. A privát kurzust csak az oktató és a beiratkozott hallgatók láthatják.

4.9.5. Prezentáció-feltöltés leckéhez rendeléssel és duplikáció-szűréssel

A `POST /courses/{id}/add-presentations` végpont két fontos funkcióval bővült: az opcionális `lessonId` paraméter lehetővé teszi, hogy a feltöltött fájl egy konkrét leckéhez is hozzárendelődjön; a duplikáció-ellenőrzés pedig megakadályozza, hogy azonos nevű fájlt kétszer töltsön fel az oktató.

Programkód 4.22. Prezentáció-feltöltés duplikáció-szűréssel és leckéhez rendeléssel

```
@PostMapping("/{id}/add-presentations")
public String addPresentations(
        @PathVariable Long id,
        @RequestParam("newPresentations") MultipartFile[] files,
        @RequestParam(required = false) Long lessonId,
        Authentication auth,
        RedirectAttributes redirectAttributes) {

    // ... jogosultság-ellenőrzés ...

    Lesson selectedLesson = null;
    if (lessonId != null) {
        selectedLesson = lessonService.findById(lessonId).orElse
            (null);
    }
```

```
    List<Presentation> existing =
            presentationService.getPresentationsByCourse(course)
               ;
    List<String> existingNames =
            existing.stream().map(Presentation::getFileName).
               toList();

    int order = existing.size() + 1;
    int addedCount = 0, skippedCount = 0;

    for (MultipartFile file : files) {
        if (!file.isEmpty()) {
            String orig  = file.getOriginalFilename();
            String lower = orig != null ? orig.toLowerCase() : "
               ";
            if (lower.endsWith(".pdf") || lower.endsWith(".ppt")
                    || lower.endsWith(".pptx")) {

                String targetName = orig;
                if (lower.endsWith(".ppt") || lower.endsWith(".
                   pptx"))
                    targetName = orig.substring(0,
                            orig.lastIndexOf('.')) + ".pdf";

                if (existingNames.contains(targetName)) {
                    skippedCount++;
                    continue;
                }

                presentationService.processAndSavePresentation(
                        file, course, selectedLesson, order++);
                addedCount++;
            }
        }
    }
    // ... visszajelző flash üzenetek ...
    return "redirect:/courses/" + id + "/manage";
}
```

A visszajelzés háromféle lehet: ha minden fájl kihagyásra kerül (már létezik), ha csak egy részük kerül feltöltésre, vagy ha mindegyik sikeresen feltöltött. Ez a részletes visszajelzés segít az oktatónak átlátni, mi történt a feltöltés során.

4.10. Prezentáció-feldolgozás megvalósítása

4.10.1. A PresentationService

A `PresentationService` feladata a feltöltött prezentációs fájlok feldolgozása és adatbázisba mentése. A `processAndSavePresentation()` metódus a feltöltött fájl kiterjesztése alapján dönt a feldolgozás módjáról: PDF fájl esetén a bájttömb közvetlenül

mentésre kerül, míg `.ppt` és `.pptx` formátum esetén a metódus először PDF-fé konvertálja a fájlt, majd az így kapott bájttömböt menti el. A prezentáció egy kurzushoz kötelezően, egy leckéhez opcionálisan kapcsolódik — utóbbi `null` értékű is lehet, ha az oktató nem rendeli konkrét leckéhez a feltöltött anyagot.

Programkód 4.23. A prezentáció-feldolgozó metódus

```
public Presentation processAndSavePresentation(
        MultipartFile file, Course course,
        Lesson lesson, Integer orderIndex) throws Exception {
    String fileName    = file.getOriginalFilename();
    byte[] fileData    = file.getBytes();
    String contentType = file.getContentType();

    if (fileName != null) {
        String lower = fileName.toLowerCase();
        if (lower.endsWith(".pptx") || lower.endsWith(".ppt")) {
            boolean isPptx = lower.endsWith(".pptx");
            fileData    = convertPptToPdfBytes(fileData, isPptx)
               ;
            fileName    = fileName.substring(0,
                            fileName.lastIndexOf('.')) + ".pdf";
            contentType = "application/pdf";
        }
    }

    Presentation p = new Presentation(
            fileName, fileData, contentType, course, lesson,
               orderIndex);
    return presentationRepository.save(p);
}
```

4.10.2. PowerPoint–PDF konverzió

A konverzió az Apache POI és az Apache PDFBox együttes alkalmazásával valósul meg. Az Apache POI két modulja eltérő formátumokat kezel: a `XMLSlideShow` osztály a modern `.pptx` (OOXML), a `HSLFSlideShow` osztály pedig a régebbi bináris `.ppt` formátumot dolgozza fel. A konverzió során a diák listáján végigiterálva minden egyes dia egy `BufferedImage`-re kerül renderelésre, amelyet az Apache PDFBox PDF-oldalként rögzít.

Programkód 4.24. Dia renderelés és PDF-összeállítás

```
private byte[] convertPptToPdfBytes(byte[] pptBytes, boolean
   isPptx)
        throws Exception {
    try (PDDocument pdfDocument = new PDDocument();
         ByteArrayInputStream  bais = new ByteArrayInputStream(
            pptBytes);
         ByteArrayOutputStream baos = new ByteArrayOutputStream
            ()) {
```

```
        Dimension pgsize;
        List<?> slides;
        if (isPptx) {
            XMLSlideShow ppt = new XMLSlideShow(bais);
            pgsize = ppt.getPageSize();
            slides = ppt.getSlides();
        } else {
            HSLFSlideShow ppt = new HSLFSlideShow(bais);
            pgsize = ppt.getPageSize();
            slides = ppt.getSlides();
        }

        int scale = 3;
        int width  = (int)(pgsize.width  * scale);
        int height = (int)(pgsize.height * scale);

        for (Object slideObj : slides) {
            BufferedImage img = new BufferedImage(
                    width, height, BufferedImage.TYPE_INT_RGB);
            Graphics2D g = img.createGraphics();

            g.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
                               RenderingHints.VALUE_ANTIALIAS_ON
                                  );
            g.setRenderingHint(RenderingHints.KEY_RENDERING,
                               RenderingHints.
                                  VALUE_RENDER_QUALITY);
            g.setRenderingHint(RenderingHints.KEY_INTERPOLATION,
                               RenderingHints.
                                  VALUE_INTERPOLATION_BICUBIC);
            g.setRenderingHint(RenderingHints.
               KEY_TEXT_ANTIALIASING,
                               RenderingHints.
                                  VALUE_TEXT_ANTIALIAS_ON);

            g.setPaint(Color.white);
            g.fill(new Rectangle2D.Float(0, 0, width, height));
            g.scale(scale, scale);

            if (isPptx) ((XSLFSlide) slideObj).draw(g);
            else        ((HSLFSlide) slideObj).draw(g);

            PDPage page = new PDPage(
                    new PDRectangle(pgsize.width, pgsize.height)
                       );
            pdfDocument.addPage(page);

            PDImageXObject pdImage =
                    JPEGFactory.createFromImage(pdfDocument, img
                       , 0.90f);
```

```
            try (PDPageContentStream cs =
                        new PDPageContentStream(pdfDocument,
                            page)) {
                cs.drawImage(pdImage, 0, 0, pgsize.width, pgsize
                    .height);
            }
        }

        pdfDocument.save(baos);
        return baos.toByteArray();
    }
}
```

A képek 3-szoros skálán készülnek, ami azt jelenti, hogy az eredeti dia koordinátarendszeréhez képest háromszor akkora felbontású `BufferedImage` jön létre. A `Graphics2D` kontextust a `g.scale(scale, scale)` hívással skálázzuk, hogy a POI renderelője az eredeti koordinátarendszerben dolgozzon. A kész képet JPEG formátumban, 90%-os minőséggel adjuk hozzá a PDFBox dokumentumhoz, miközben a PDF-oldal mérete az eredeti diával megegyező marad. Ez biztosítja, hogy a böngésző éles, jól olvasható képet jelenítsen meg.

4.10.3. Prezentáció streamelése

A `PresentationController` a tárolt PDF bájttömböt `ResponseEntity<byte[]>` formában streameli a böngésző felé. A `@Transactional` annotáció megakadályozza a `LazyInitializationException` hibát a lusta betöltésű `lesson` kapcsolat hozzáférésekor. A hozzáférés-ellenőrzés figyelembe veszi a kurzus láthatóságát is: nyilvános kurzus esetén bármely bejelentkezett felhasználó megtekintheti a prezentációt beiratkozás nélkül is, privát kurzusnál azonban csak az oktató és a beiratkozott hallgatók férhetnek hozzá.

Programkód 4.25. PDF tartalom streamelése és hozzáférés-ellenőrzés

```
@Controller
@Transactional
public class PresentationController {

    private boolean hasAccessToPresentation(
            Presentation presentation, Authentication auth) {
        if (auth == null) return false;
        Optional<User> userOpt =
                userService.getUserByUsername(auth.getName());
        if (userOpt.isEmpty()) return false;

        User   user   = userOpt.get();
        Course course = presentation.getCourse();

        if (user.getRole() == Role.ADMIN) return true;

        if (course.getInstructor() != null
                && course.getInstructor().getId().equals(user.
                    getId()))
```

```
            return true;

        if (Boolean.TRUE.equals(course.getIsPublic())) return
           true;

        return course.getEnrolledUsers().contains(user);
    }
}
```

A streamelési végpont `ContentDisposition.inline()` fejlécet állít be, amely a böngészőt a beépített PDF-megjelenítő használatára ösztönzi letöltés helyett. Ha a fájl bájttömb üres vagy hiányzik, a végpont `404 Not Found` választ ad vissza.

4.11. Kvízrendszer megvalósítása

4.11.1. Kvíz kitöltési és kiértékelési logika

A `GET /courses/{courseId}/quizzes/{quizId}/take` végpont induláskor ellenőrzi, hogy a hallgató korábban már kitöltötte-e a kvízt, és az egyszeri kitöltés be van-e kapcsolva. Ha igen, az előző eredmény kerül megjelenítésre.

A `POST /courses/{courseId}/quizzes/{quizId}/submit` végpont szerver oldalon értékeli ki az eredményt: a beküldött válaszokat (`question_{id}` paraméterneveken) összehasonlítja a tárolt helyes válaszokkal, kiszámolja a százalékos teljesítményt, és a `Quiz.calculateGrade()` metódussal meghatározza az érdemjegyet.

4.11.2. Ponthatár-módosítás visszamenőleges hatása

A `POST /courses/{courseId}/quizzes/{quizId}/thresholds` végpont az új ponthatárok elmentése után az összes korábbi `QuizResult` osztályzatát újraszámítja, majd `saveAll()` hívással egyetlen tranzakcióban menti a változásokat.

Programkód 4.26. Visszamenőleges osztályzat-újraszámítás ponthatár-módosítás után

```
quiz.setExcellentThreshold(excellentThreshold);
quiz.setGoodThreshold(goodThreshold);
quiz.setAverageThreshold(averageThreshold);
quiz.setPassingThreshold(passingThreshold);
quiz.setAllowMultipleAttempts(allowMultipleAttempts);
quizService.saveQuiz(quiz);

List<QuizResult> results = quizResultRepository.findByQuiz(quiz)
    ;
for (QuizResult r : results) {
    r.setGrade(quiz.calculateGrade(r.getPercentage()));
}
quizResultRepository.saveAll(results);
```

4.12. E-mail értesítési rendszer

Az `EmailService` öt HTML-alapú e-mail típust kezel: regisztrációs üdvözlőlevelet, jelszóváltoztatási értesítőt, felhasználónév-módosítási értesítőt, e-mail-változtatási értesítőt, és fióktörlési visszaigazolást. Az e-mail törzsek Java szövegtömbökkel (*text blocks*) és `String.formatted()` behelyettesítéssel épülnek fel. Az e-mail küldési hiba szándékosan csak naplózásra kerül, és nem propagálódik, hogy SMTP-hiba ne akassza meg a felhasználói folyamatot.

4.13. Nézetek és sablon-rendszer

4.13.1. Sablon-struktúra

A nézetek az `src/main/resources/templates/` könyvtárban helyezkednek el, funkcionális területenként alkönyvtárakba szervezve:

- Gyökérszintű sablonok: `index.html`, `login.html`, `register.html`, `profile.html`, `about.html`.
- `courses/`: `list.html`, `view.html`, `create.html`, `manage.html`, `viewer.html` (diavetítő), `take-quiz.html`, `quiz-result.html`, `quiz-results.html`, `quiz-thresholds.html`, `create-quiz.html`.
- `dashboard/`: `admin.html`, `instructor.html`, `student.html`.
- `presentation/`: `viewer.html`.
- `fragments/`: `sidebar.html`.
- `error/`: `404.html`, `error.html`.

4.13.2. Mobiloptimalizálás és reszponzív nézetek

Az optimalizálás során az összes sablon reszponzív megjelenítésre lett felkészítve. A Bootstrap keretrendszer rácsrendszere (`col-md-*`, `col-sm-*` osztályok) biztosítja, hogy a tartalom asztali, tabletes és mobilos képernyőn egyaránt olvasható elrendezést vegyen fel. A navigációs sáv mobilon összecsukható hamburger-menüként jelenik meg.

A kurzuslista kártya-alapú elrendezést (`card`) alkalmaz, amelyek mobilon teljes szélességűek, asztali nézetben pedig többoszlopos rácsban helyezkednek el. A kvízkitöltő és az eredményoldal gombjainak mérete és tapintható területe is mobil-barát értékekre lett beállítva.

4.13.3. Thymeleaf–Spring Security integráció

A `thymeleaf-extras-springsecurity6` függőség aktiválja a `sec:` névteret, amely HTML-szinten teszi lehetővé a szerepkörfüggő megjelenítést:

Programkód 4.27. Szerepkörfüggő megjelenítés Thymeleaf sablonban

```
<div sec:authorize="hasRole('INSTRUCTOR')">
    <a th:href="@{/courses/create}">Kurzus létrehozása</a>
```

```
</div>

<span sec:authorize="isAuthenticated()"
      th:text="${#authentication.name}">
    Felhasználónév
</span>
```

Ez a megközelítés kizárólag megjelenítési szintű elrejtést valósít meg; a tényleges hozzáférés-védelmet minden esetben a Spring Security szűrőlánc és a service-szintű ellenőrzés biztosítja.

4.13.4. Statikus erőforrások

A `static/css/` könyvtár két stíluslapot tartalmaz: a `main.css` az összes hitelesített oldal egységes vizuális stílusát határozza meg, a `landing.css` pedig a nyilvánosan elérhető főoldal egyedi megjelenését — ez megakadályozza, hogy a nyilvános oldal stílusai felülírják az alkalmazás belső stílusait.

5. fejezet

Tesztelés

5.1. Tesztelési stratégia

Az alkalmazás tesztelése két egymást kiegészítő megközelítéssel valósult meg: automatizált Spring Boot integrációs teszteléssel és részletes manuális funkcionális teszteléssel. A tesztelés az alábbi területeket fedte le:

- a Spring alkalmazáskontextus helyes betöltése (konfigurációs, bean-definíciós hibák kizárása),
- szerepkör-alapú hozzáférés-vezérlés helyessége minden URL-útvonalon,
- a kurzus-, lecke- és kvízkezelési műveletek teljes életciklusa,
- a prezentáció-feltöltési, konverziós és duplikáció-szűrési folyamat,
- a regisztrációs és bejelentkezési folyamat, beleértve a validációs szabályokat,
- mobilos és asztali reszponzív megjelenítés.

Az integrációs tesztek a teljes Spring-kontextust töltik be, ezáltal a rétegek közötti interakciókat is ellenőrzik, nem csupán az egyes osztályokat izoláltan. A manuális tesztelés az automatizált tesztekkel nehezebben lefedhetős felhasználói felületi és üzleti folyamatokra fókuszál.

5.2. Automatizált tesztek

5.2.1. Alkalmazáskontextus-teszt

Az `ElearningApplicationTests` osztály az alkalmazás alapvető egészségét ellenőrzi: induláskor betölti-e a Spring IoC-konténer az összes bean-t hibák nélkül. Ez a teszt a legfontosabb regressziós háló: bármely konfigurációs hiba, körkörös függőség (*circular dependency*) vagy hiányzó bean-definíció azonnal láthatóvá válik.

Programkód 5.1. Spring kontextus betöltési teszt

```
@SpringBootTest
class ElearningApplicationTests {
```

```
    @Test
    void contextLoads() {
    }
}
```

A `@SpringBootTest` annotáció az alkalmazás teljes kontextusát indítja el, beleértve a biztonsági konfigurációt, az adatbázis-inicializálást és az összes service bean-t. Sikeres lefutása garantálja, hogy az alkalmazás éles körülmények között is elindítható.

5.2.2. Regisztrációs és bejelentkezési integrációs teszt

Az `AuthControllerTest` osztály a regisztrációs és bejelentkezési folyamat HTTP-szintű viselkedését ellenőrzi a Spring MVC `MockMvc` tesztelési keretrendszerével.

Programkód 5.2. Bejelentkezési és regisztrációs MockMvc tesztek

```
@SpringBootTest
@AutoConfigureMockMvc
class AuthControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @Test
    void loginPage_shouldReturn200() throws Exception {
        mockMvc.perform(get("/login"))
                .andExpect(status().isOk())
                .andExpect(view().name("login"));
    }

    @Test
    void registerPage_shouldReturn200() throws Exception {
        mockMvc.perform(get("/register"))
                .andExpect(status().isOk())
                .andExpect(view().name("register"));
    }

    @Test
    void register_withValidData_shouldRedirect() throws
       Exception {
        mockMvc.perform(post("/register")
                    .param("username", "testuser")
                    .param("email", "test@example.com")
                    .param("password", "password123")
                    .param("firstName", "Test")
                    .param("lastName", "User")
                    .with(csrf()))
                .andExpect(status().is3xxRedirection())
                .andExpect(redirectedUrl("/login"));
    }

    @Test
```

```
    void register_withDuplicateUsername_shouldShowError()
            throws Exception {
        mockMvc.perform(post("/register")
                    .param("username", "duplicate")
                    .param("email", "dup1@example.com")
                    .param("password", "pass1234")
                    .with(csrf()));

        mockMvc.perform(post("/register")
                    .param("username", "duplicate")
                    .param("email", "dup2@example.com")
                    .param("password", "pass1234")
                    .with(csrf()))
                .andExpect(status().isOk())
                .andExpect(model().attributeExists("error"));
    }
}
```

A `with(csrf())` segédfüggvény gondoskodik arról, hogy az állapotváltoztató POST kérések tartalmazzák a szükséges CSRF-tokent.

5.2.3. Jogosultságvezérlési integrációs teszt

A `SecurityIntegrationTest` osztály ellenőrzi, hogy a különböző szerepkörű felhasználók csak az engedélyezett URL-ekhez férnek hozzá. A tesztek a `@WithMockUser` annotáció segítségével szintetikus hitelesített felhasználóként futnak, valódi bejelentkezési folyamat nélkül.

Programkód 5.3. Szerepkör-alapú hozzáférés-vezérlés integrációs tesztje

```
@SpringBootTest
@AutoConfigureMockMvc
class SecurityIntegrationTest {

    @Autowired
    private MockMvc mockMvc;

    @Test
    void protectedRoute_withoutAuth_shouldRedirectToLogin()
            throws Exception {
        mockMvc.perform(get("/courses"))
                .andExpect(status().is3xxRedirection())
                .andExpect(redirectedUrlPattern("**/login"));
    }

    @Test
    @WithMockUser(roles = "STUDENT")
    void courseCreate_asStudent_shouldBeForbidden() throws
       Exception {
        mockMvc.perform(get("/courses/create"))
                .andExpect(status().isForbidden());
    }
```

```
    @Test
    @WithMockUser(roles = "INSTRUCTOR")
    void courseCreate_asInstructor_shouldBeAllowed() throws
       Exception {
        mockMvc.perform(get("/courses/create"))
               .andExpect(status().isOk());
    }

    @Test
    @WithMockUser(roles = "STUDENT")
    void adminPanel_asStudent_shouldBeForbidden() throws
       Exception {
        mockMvc.perform(get("/admin/users"))
               .andExpect(status().isForbidden());
    }

    @Test
    @WithMockUser(roles = "ADMIN")
    void adminPanel_asAdmin_shouldBeAllowed() throws Exception {
        mockMvc.perform(get("/admin/users"))
               .andExpect(status().isOk());
    }
}
```

5.2.4. Kvíz üzleti logika egységteszt

A `QuizServiceTest` osztály a kvízkiértékelési logikát ellenőrzi izoláltan, Spring-kontextus betöltése nélkül. Ez gyors, az adatbázistól független visszajelzést ad a pontszámítás és az osztályzatkiosztás helyességéről.

Programkód 5.4. Kvíz pontszámítás és osztályzatkiosztás egységtesztje

```
class QuizServiceTest {

    @Test
    void calculateGrade_above85_shouldReturn5() {
        Quiz quiz = buildQuiz(85, 70, 55, 40);
        assertEquals(5, quiz.calculateGrade(90));
    }

    @Test
    void calculateGrade_below40_shouldReturn1() {
        Quiz quiz = buildQuiz(85, 70, 55, 40);
        assertEquals(1, quiz.calculateGrade(30));
    }

    @Test
    void calculateGrade_exactly85_shouldReturn5() {
        Quiz quiz = buildQuiz(85, 70, 55, 40);
        assertEquals(5, quiz.calculateGrade(85));
```

```
    }

    @Test
    void calculateGrade_between70and84_shouldReturn4() {
        Quiz quiz = buildQuiz(85, 70, 55, 40);
        assertEquals(4, quiz.calculateGrade(75));
    }

    private Quiz buildQuiz(int excellent, int good,
                           int average, int pass) {
        Quiz q = new Quiz();
        q.setExcellentThreshold(excellent);
        q.setGoodThreshold(good);
        q.setAverageThreshold(average);
        q.setPassingThreshold(pass);
        return q;
    }
}
```

A tesztek a `Quiz.calculateGrade()` metódust közvetlenül hívják — Spring-kontextus nélkül is futtathatók, mert az osztályzatszámítási logika az entitásosztályban van, nem service-ben. Az egységtesztek szimmetrikusan lefedik a határértékeket (pontosan 85%), a közbülső értékeket és az alsó tartomány elégtelen esetét.

5.3. Manuális funkcionális tesztelés

A manuális tesztelés a beágyazott H2 adatbázist használó, helyi fejlesztői szerveren futó alkalmazáson történt. Az alkalmazás a `DataInitializer` által létrehozott tesztfelhasználókkal rendelkezett: `admin`, `instructor`, `kovacs.janos`, `nagy.anna` és `student`.

5.3.1. Hitelesítési tesztek

5.1. táblázat: Hitelesítési tesztesetek és eredményeik

Teszteset leírása	**Bemenet / körülmény**	**Várt eredmény**	**Eredmény**
Bejelentkezés helyes adatokkal (hallgató)	`student` / helyes jelszó	Átirányítás a hallgató irányítópultra	✓
Bejelentkezés helyes adatokkal (oktató)	`instructor` / helyes jelszó	Átirányítás az oktató irányítópultra	✓
Bejelentkezés helyes adatokkal (admin)	`admin` / helyes jelszó	Átirányítás az admin irányítópultra	✓
Bejelentkezés helytelen jelszóval	Érvényes username, rossz jelszó	Hibaüzenet a bejelentkezési oldalon	✓
Bejelentkezés nem létező felhasználóval	Nem létező username	Hibaüzenet a bejelentkezési oldalon	✓
Regisztráció érvényes adatokkal	Egyedi username és e-mail	Átirányítás a bejelentkezési oldalra	✓
Regisztráció foglalt felhasználónévvel	Már létező username	Hibaüzenet, az űrlap kitöltve marad	✓
Regisztráció foglalt e-mail címmel	Már létező e-mail	Hibaüzenet, az űrlap kitöltve marad	✓
Regisztráció hiányos adatokkal	Üres kötelező mező	Validációs hibaüzenet	✓
Kijelentkezés	Bejelentkezett felhasználó	Átirányítás a főoldalra	✓

5.3.2. Kurzus- és leckekezelési tesztek

5.2. táblázat: Kurzus- és leckekezelési tesztesetek

Teszteset leírása	Végrehajtó	Várt eredmény	Eredmény
Nyilvános kurzus létrehozása	Oktató	Sikeres mentés, megjelenik a listában	✓
Privát kurzus létrehozása	Oktató	Csak az oktatónak látható	✓
Saját kurzus törlése	Oktató	Sikeres törlés	✓
Más oktató kurzusának törlési kísérlete	Oktató	Hozzáférés megtagadva	✓
Lecke hozzáadása kurzushoz	Oktató	Lecke megjelenik a kezelőfelületen	✓
Leckék sorrendben jelennek meg	Hallgató	Orderindex szerint növekvő sorrend	✓
Hallgató hozzáadása kurzushoz	Oktató	Hallgató megjelenik a résztvevők között	✓
Beiratkozás nyilvános kurzusra	Hallgató	Sikeres beiratkozás	✓
Beiratkozás privát kurzusra (önálló)	Hallgató	Hozzáférés megtagadva	✓
Kurzus törlése admin által	Admin	Sikeres törlés	✓
Kurzuslétrehozás kísérlete hallgatóként	Hallgató	403 válasz	✓
Kurzuskeresés kulcsszóval (cím)	Bármely	Csak illeszkedő kurzusok látszanak	✓
Kurzuskeresés kulcsszóval (leírás)	Bármely	Csak illeszkedő kurzusok látszanak	✓
Üres keresési mező	Bármely	Összes elérhető kurzus látszódik	✓

Kiemelt megállapítás a leckekezelésről: a leckék az `orderIndex` mező alapján rendezetten jelennek meg a kurzusnézeten, és a hallgatók leckénként is böngészhetnek a hozzájuk csatolt prezentációk között. A lecke létrehozásakor az oktató megadja a sorrendi indexet, amelyet a `LessonRepository` automatikusan rendez a lekérdezés során.

5.3.3. Prezentáció-feltöltési tesztek

5.3. táblázat: Prezentáció-feltöltési tesztesetek és eredményeik

Teszteset leírása	**Fájltípus**	**Várt eredmény**	**Eredmény**
PowerPoint feltöltése (.pptx)	`.pptx`	Sikeres konverzió, PDF tárolás	✓
PowerPoint feltöltése (.ppt)	`.ppt`	Sikeres konverzió, PDF tárolás	✓
PDF feltöltése	`.pdf`	Közvetlen tárolás, konverzió nélkül	✓
Feltöltés leckéhez rendeléssel	`.pdf`	Lecke-szintű megjelenítés	✓
Feltöltés kurzusszinten (lecke nélkül)	`.pdf`	Kurzusszintű megjelenítés	✓
Azonos nevű fájl újbóli feltöltése	Bármely	Kihagyás, tájékoztató üzenet	✓
Részleges duplikátum (egy kihagyva, egy feltöltve)	Vegyes	Vegyes visszajelzés	✓
Nyilvános kurzus prezentációja (bejelentkezett, nem beiratkozott)	—	PDF-nézegető betöltődik	✓
Privát kurzus prezentációja (nem beiratkozott)	—	Hozzáférés megtagadva	✓
Nem támogatott fájlformátum feltöltése	`.docx`	Hibaüzenet, feltöltés visszautasítva	✓
50 MB-nál nagyobb fájl feltöltése	Bármely	Hibaüzenet a méretkorlátról	✓
Prezentáció törlése	—	Eltűnik a listáról	✓

A duplikáció-szűrési logika tesztelésekor azt vizsgáltuk, hogy az oktató három fájlt tölt fel egyszerre, amelyek közül kettő már létezik a kurzusban. Az alkalmazás az egy új fájlt feltöltötte, a két már meglévőt kihagyta, és részletes visszajelzést adott: *„1 fájl sikeresen feltöltve. 2 fájl kihagyva, mert már létezik."*

A nyilvános kurzus prezentáció-elérési tesztjénél egy be nem iratkozott, de bejelentkezett hallgató is sikeresen meg tudta tekinteni a nyilvános kurzus prezentációját. Ez az optimalizálás során bevezetett változtatás összhangban van azzal, hogy a nyilvános kurzus tartalmát bárki megtekintheti — beiratkozás a kvízek kitöltéséhez szükséges.

5.3.4. Kvíztesztelés

5.4. táblázat: Kvíz életciklus tesztesetek és eredményeik

Teszteset leírása	**Végrehajtó**	**Várt eredmény**	**Eredmény**
Kvíz létrehozása kérdésekkel	Oktató	Kvíz megjelenik a kurzuson	✓
Kvíz kitöltése (minden helyes)	Hallgató	100%, jeles osztályzat	✓
Kvíz kitöltése (részben helyes)	Hallgató	Arányos pontszám és osztályzat	✓
Kvíz kitöltése (minden helytelen)	Hallgató	0%, elégtelen osztályzat	✓
Ismételt kitöltési kísérlet (ha tiltott)	Hallgató	Előző eredmény jelenik meg	✓
Ismételt kitöltési kísérlet (ha engedélyezett)	Hallgató	Kitöltési form újra megjelenik	✓
Eredmény megtekintése az oktatónak	Oktató	Összesített statisztikák helyesek	✓
Ponthatárok módosítása kitöltések után	Oktató	Korábbi osztályzatok automatikusan frissülnek	✓
Kvíz megtekintési kísérlet nem beiratkozott hallgatóval	Hallgató	Hozzáférés megtagadva	✓

5.3.5. Irányítópult tesztek

5.5. táblázat: Irányítópult-tesztesetek és eredményeik

Teszteset leírása	Végrehajtó	Várt eredmény	Eredmény
Admin irányítópult megnyitása	Admin	Felhasználók és kurzusok statisztikái látszanak	✓
Oktató irányítópult megnyitása	Oktató	Saját kurzusok listája és statisztikák	✓
Hallgató irányítópult megnyitása	Hallgató	Beiratkozott és elérhető kurzusok listája	✓
Hallgató irányítópultján az elérhető lista nem tartalmaz beiratkozott kurzust	Hallgató	Beiratkozott kurzus eltűnik az elérhetők közül	✓
Admin töröl felhasználót az irányítópultról	Admin	Felhasználó törlődik, visszajelzés megjelenik	✓
Admin töröl kurzust az irányítópultról	Admin	Kurzus törlődik, statisztikák frissülnek	✓

5.3.6. Felhasználókezelési tesztek

5.6. táblázat: Adminisztrátori felhasználókezelési tesztesetek

Teszteset leírása	Végrehajtó	Várt eredmény	Eredmény
Összes felhasználó listázása	Admin	Minden regisztrált felhasználó látható	✓
Felhasználó szerepkörének módosítása	Admin	Az új szerepkör azonnal érvényes	✓
Felhasználó törlése	Admin	Fiók és beiratkozások törlődnek	✓
Saját fiók törlési kísérlete adminként	Admin	Hibaüzenet, törlés megakadályozva	✓
Admin felület elérési kísérlete oktató által	Oktató	403 válasz, hozzáférés megtagadva	✓

5.3.7. E-mail értesítési teszt

A regisztrációs e-mail küldés tesztelésekor valódi Gmail SMTP-konfigurációval ellenőriztük, hogy az `EmailService` helyesen küld-e üdvözlő e-mailt az újonnan regisztrált felhasználó megadott e-mail-címére. Az e-mail megérkezett és tartalma helyes volt. Az érzékeny konfigurációs adatok az `application.properties` fájlban helyezkednek el.

5.4. Biztonsági tesztelés

5.4.1. URL-manipulációs kísérletek

A biztonsági tesztelés folyamán kísérletet tettem a jogosultságvezérlés manuális, URL-manipulációval történő megkerülésére:

- **Hitelesítés nélküli hozzáférés:** A `/courses`, `/courses/1/manage` és `/dashboard` URL-ek bejelentkezés nélkül mindegyike a `/login` oldalra irányított át.
- **Kereszt-szerepkörű hozzáférés:** Hallgató szerepkörű felhasználóval a `/courses/create` és `/courses/1/manage` oldalak mindegyike 403 Forbidden választ adott vissza.
- **Leckelétrehozás közvetlen URL-lel:** Hallgató szerepkörű felhasználóval a `POST /lessons/create` végpontot hívtuk meg. A Spring Security az URL-szintű védelem hiánya ellenére a CSRF-tokenen keresztül és a kurzustulajdonos-ellenőrzéssel megakadályozta a jogosulatlan leckét létrehozást.
- **Más oktató kurzusának szerkesztése:** A `kovacs.janos` felhasználóval az `nagy.anna` kurzusának kezelési URL-jét hívtuk meg. A service-szintű `isOwnerOrAdmin()` ellenőrzés megakadályozta a hozzáférést.

5.4.2. CSRF-védelem ellenőrzése

Egy külső HTML-oldal segítségével CSRF-token nélküli POST kérést küldtünk a kurzustörlési és leckét-létrehozási végpontokra. Az alkalmazás minden ilyen kísérletet 403 Forbidden hibával utasított vissza.

5.5. Reszponzivitás és böngészőkompatibilitás tesztelése

A fejlesztési folyamat részeként minden sablon reszponzív kialakítást kapott. A tesztelést különböző böngészőkben és képernyőméreteken lett elvégezve, beleértve a böngészők fejlesztői eszközeiben elérhető eszközszimulációs nézetet.

5.7. táblázat: Böngészőkompatibilitási és reszponzív tesztelés összefoglalója

Böngésző és verzió	Felbontás / eszköz	Eredmény
Google Chrome 124	1920×1080 (asztali)	✓
Google Chrome 124	390×844 (mobil szimuláció)	✓
Google Chrome 124	768×1024 (tablet szimuláció)	✓
Mozilla Firefox 125	1920×1080 (asztali)	✓
Mozilla Firefox 125	390×844 (mobil szimuláció)	✓
Microsoft Edge 124	1920×1080 (asztali)	✓

Mobilos nézetben a következő elemek viselkedését ellenőriztük külön:

- **Navigáció:** Az összecsukható hamburger-menü minden tesztelt böngészőben megfelelően nyílt és csukódott.
- **Kurzuslista:** A kártyák mobilon teljes szélességűek lesznek, tablet nézetben két, asztali nézetben három oszlopban helyezkednek el.
- **Kvízkitöltő:** A rádiógombok és a beküldő gomb tapintható területe mobilon megfelelő méretű volt.
- **Keresőmező:** A kurzuslista keresőmezője mobilon is teljes szélességű.

5.6. Ismert korlátok és fejlesztési lehetőségek

5.6.1. Ismert korlátok

- **Memória-adatbázis:** A fejlesztési konfigurációban H2 memória-adatbázis kerül alkalmazásra, ezért az alkalmazás leállításakor az adatok elvesznek. Éles üzemeltetéshez PostgreSQL vagy MySQL adatbázis szükséges, amelyhez csak az `application.properties` és a `pom.xml` módosítása szükséges.
- **Fájladatok az adatbázisban:** A feltöltött prezentációs fájlok bináris tartalma (`byte[]`) az adatbázisban tárolódik. Ez nagyméretű fájlok esetén az adatbázis méretét jelentősen növeli és lassíthatja a lekérdezéseket.
- **Nincs automatizált teszt a prezentáció-konverzióhoz:** Az Apache POI és PDFBox alapú konverziós logika kizárólag manuálisan tesztelésre kerül; automatizált egységteszt jelenleg nem biztosít regresszióvédelmet ezen a területen.
- **Egyszeri szerepkör:** Egy felhasználó egyszerre csak egyetlen szerepkörrel rendelkezhet, így egy oktató nem iratkozhat be más oktatók kurzusaira hallgatóként.
- **Leckesorrend módosítása:** A leckék sorrendi indexét egyelőre csak manuálisan, kézzel megadott értékkel lehet beállítani; drag-and-drop rendezési felület nem érhető el.
- **Alapszintű statisztikák:** A kvíz eredménystatisztikák jelenleg átlagot, kísérletszámot és átmenési arányt tartalmaznak; kérdésenkénti elemzés nem érhető el.

5.6.2. Fejlesztési lehetőségek

- **Külső fájltároló integráció:** A prezentációs fájlok áthelyezése fájlrendszer-alapú vagy objektumtároló megoldásba csökkentené az adatbázis méretét és javítaná a lekérési teljesítményt.
- **Leckesorrend drag-and-drop szerkesztése:** JavaScript alapú fogd-és-vidd felület az `orderIndex` értékek vizuális módosításához.
- **REST API réteg:** A Thymeleaf-alapú szerver oldali renderelés mellé egy REST API réteg lehetővé tenné mobilalkalmazásokkal való integrációt.
- **Automatizált end-to-end tesztelés:** Selenium vagy Playwright alapú böngészővezérelt tesztek a felhasználói felület regressziós tesztelésére, beleértve a mobilos megjelenítés ellenőrzését is.
- **Kérdésenkénti statisztikák:** A helyes arány kérdésenként megmutatná, mely kérdések bizonyulnak a hallgatók számára a legnehezebbeknek.
- **Dedikált tesztelési profil:** Külön `application-test.properties` Spring-profil bevezetése az automatizált tesztek számára, elkülönítve a fejlesztési és tesztelési beállításokat.

6. fejezet

Összefoglalás

A dolgozat egy Java alapú e-learning webalkalmazás — a ME Learning platform — tervezését és megvalósítását mutatta be. Az alkalmazás Spring Boot 3.x keretrendszerre épül, Thymeleaf sablon-motorral és H2 memória-adatbázissal, és a Miskolci Egyetem hallgatói számára nyújt kurzuskezelési, prezentáció-megjelenítési és kvíz-alapú tudásellenőrzési funkciókat.

A fejlesztés során a következő főbb eredmények születtek:

- Szerepkör-alapú hozzáférés-vezérlés megvalósítása három szerepkörrel (adminisztrátor, oktató, hallgató), Spring Security segítségével.
- Kurzuskezelő rendszer, amely lehetővé teszi kurzusok létrehozását, módosítását, hallgatók beiratkoztatását és eltávolítását.
- Prezentáció-feldolgozó modul, amely PowerPoint fájlokat (.ppt, .pptx) automatikusan PDF formátumba konvertál, magas felbontásban renderelve az egyes diákat az Apache POI és PDFBox könyvtárak segítségével.
- Kvízrendszer, amely az oktatók által létrehozott teszteket kezeli, eredményeket rögzít, statisztikákat számít, és visszamenőleges osztályzat-újraszámítást tesz lehetővé.
- E-mail értesítési funkció Gmail SMTP-n keresztül.
- Reszponzív, felhasználóbarát felhasználói felület Thymeleaf sablonokkal és egyedi CSS stíluslapokkal.

Az alkalmazás a kitűzött funkcionális követelményeket teljesíti. A manuális tesztelés során az összes azonosított teszteset sikeresen lefutott. A rendszer könnyen bővíthető: a H2 adatbázis éles környezetben PostgreSQL-re cserélhető, és az architektúra lehetővé teszi REST API réteg hozzáadását is.

A fejlesztés során szerzett tapasztalatok alapján a jövőbeli fejlesztési irányok között szerepel a fájlok adatbázison kívüli tárolása, a részletesebb teljesítménystatisztikák bevezetése, valamint a mobilalkalmazás-integráció lehetősége.

Források

[1] I. Elaine Allen és Jeff Seaman. „Changing Course: Ten Years of Tracking Online Education in the United States". *Babson Survey Research Group* (2013).

[2] Apache Software Foundation. *Apache PDFBox – A Java PDF Library.* `https://pdfbox.apache.org`. Accessed: 2024. 2023.

[3] Apache Software Foundation. *Apache POI – the Java API for Microsoft Documents.* `https://poi.apache.org`. Accessed: 2024. 2023.

[4] Axim Collaborative. *Open edX Platform.* `https://openedx.org`. Accessed: 2024. 2023.

[5] Christian Bauer, Gavin King és Gary Gregory. *Java Persistence with Hibernate.* 2nd. Shelter Island, NY: Manning Publications, 2015.

[6] Lawrence Chung és tsai. *Non-Functional Requirements in Software Engineering.* Boston, MA: Springer, 2000.

[7] Jason Cole és Helen Foster. *Using Moodle: Teaching with the Popular Open Source Course Management System.* 2nd. Sebastopol, CA: O'Reilly Media, 2007.

[8] Bruce Eckel. *Thinking in Java.* 4th. Upper Saddle River, NJ: Prentice Hall, 2006.

[9] Daniel Fernández. *Thymeleaf – Natural Templating for Java.* `https://www.thymeleaf.org`. Accessed: 2024. 2023.

[10] Google LLC. *Google Classroom.* `https://edu.google.com/products/classroom`. Accessed: 2024. 2023.

[11] Jan Groen és Maaike van der Berg. „GDPR compliance for educational platforms: Challenges and solutions". *Computer Law & Security Review* 36 (2020), 105389. old.

[12] Charles Hodges és tsai. „The Difference Between Emergency Remote Teaching and Online Learning". *EDUCAUSE Review* 27 (2020).

[13] Instructure Inc. *Canvas LMS by Instructure.* `https://www.instructure.com/canvas`. Accessed: 2024. 2023.

[14] Rod Johnson. *Expert One-on-One J2EE Design and Development.* Indianapolis, IN: Wrox Press, 2004.

[15] Moodle HQ. *Moodle Statistics.* `https://stats.moodle.org`. Accessed: 2024. 2024.

[16] Niels Provos és David Mazières. „A Future-Adaptable Password Scheme". *Proceedings of the USENIX Annual Technical Conference.* USENIX. 1999.

[17] Marc J. Rosenberg. *E-Learning: Strategies for Delivering Knowledge in the Digital Age.* New York: McGraw-Hill, 2001.

[18] Ian Sommerville. *Software Engineering*. 10th. Harlow, UK: Pearson, 2016.

[19] Laurentiu Spilcă. *Spring Security in Action*. 2nd. Shelter Island, NY: Manning Publications, 2023.

[20] Craig Walls. *Spring in Action*. 5th. Shelter Island, NY: Manning Publications, 2019.

[21] William R. Watson és Sunnie Lee Watson. „Who or What Is the Learning Management System?”: *Journal of Educational Technology & Society* 10.4 (2007), 33–40. old.

CD Használati útmutató

Ennek a címe lehet például *A mellékelt CD tartalma* vagy *Adathordozó használati útmutató* is.

Ez jellemzően csak egy fél-egy oldalas leírás. Arra szolgál, hogy ha valaki kézhez kapja a szakdolgozathoz tartozó CD-t, akkor tudja, hogy mi hol van rajta. Jellemzően elég csak felsorolni, hogy milyen jegyzékek vannak, és azokban mi található. Az elkészített programok telepítéséhez, futtatásához tartozó instrukciók kerülhetnek ide.

A CD lemezre mindenképpen rá kell tenni

- a dolgozatot egy `dolgozat.pdf` fájl formájában,
- a LaTeX forráskódját a dolgozatnak,
- az elkészített programot, fontosabb futási eredményeket (például ha kép a kimenet),
- egy útmutatót a CD használatához (ami lehet ez a fejezet külön PDF-be vagy MarkDown fájlként kimentve).